

## Divide and Conquer Methodology

It is a strategy of solving a large problem by

- breaking problem into smaller sub-problems.
- solving sub-problems.
- combining them to get desired output.

This make use of recursion.

### Steps:

1. Divide: divide given problem into subproblems using recursion.
2. Conquer: Solve smaller sub-problems recursively. If the subproblem is small enough, then solve it directly.
3. Combine: combine solutions of sub-problems that are part of recursive process to solve actual problem.

### Finding maximum and minimum

To find min. and max. numbers in a given array, first we are representing naive method and then (we) will present divide and conquer approach.

#### 1) Naive Method:

It is a basic method to solve any problem. In this method, max. and min. no. can be found separately. To find max & min. no. the following straight forward algorithm can be used:

Algorithm:  
max-min-element(numbers[])  
max := numbers[1]  
min := numbers[1]  
for i = 2 to n do  
  if numbers[i] > max then  
    max := numbers[i]  
  if numbers[i] < min then  
    min := numbers[i]

return(max, min).



### Analysis:

No. of comparisons in Naive method =  $2n-2$ .

This can be reduced by using divide and conquer approach.

### 2) Divide and conquer approach:

In this array is divided into 2 halves. Then using recursive approach max. and min. members in each halves are found, later return max. of 2 maxima of each half and min. of 2 minima of each half. Let no. of elements in array is  $y-x+1$ , where  $y$  is  $\geq x$ .

Max-Min( $x, y$ ) - return max. & min. values of an array numbers [ $x \dots y$ ].

### Algorithm:

Max-min( $x, y$ )

if  $y - x \leq 1$  then

return (max(numbers [ $x$ ], numbers [ $y$ ]), min(numbers [ $x$ ], numbers [ $y$ ]))

else

(max1, min1) = maxmin( $x$ ,  $\lfloor ((x+y)/2) \rfloor$ )

(max2, min2) = maxmin( $\lfloor ((x+y)/2) \rfloor + 1$ ,  $y$ )

return (max(max1, max2), min(min1, min2))

### Analysis:

Let  $T(n)$  be no. of comparisons made by Max-min( $x, y$ ) where no. of elements  $n = y - x + 1$

Then Recurrence relation

$$T(n) = \begin{cases} T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 2 & \text{for } n > 2 \\ 1 & \text{for } n = 2 \\ 0 & \text{for } n = 1 \end{cases}$$

Assume  $n = 2^k$ ;  $k \rightarrow$  height of recursion tree.



$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + 2 = 2 \left( 2 \cdot T\left(\frac{n}{4}\right) + 2 \right) + 2 \dots = \frac{3n}{2} - 2$$

Compared to Naive, in this divide & conquer approach no. of comparisons is less.  
Asymptotic notation of both represented by  $O(n)$ .

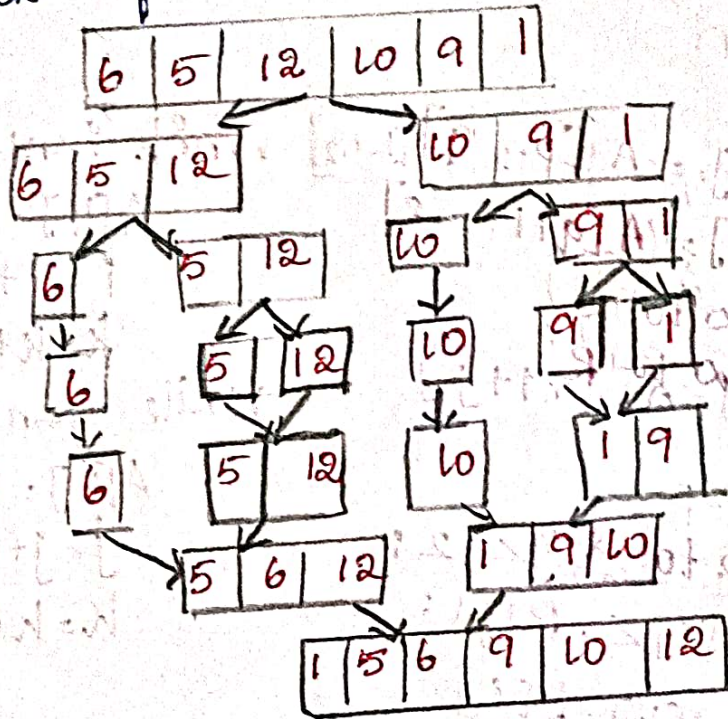
## Merge Sort

Merge Sort

It is one of the most popular sorting algorithms that is based on the principle of divide and conquer strategy. Here problem is divided into multiple sub-problem. Each sub-problem is solved individually. Finally sub-problems combined to form final solution.

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|

Example:



### Divide and conquer strategy:

Divide and conquer strategy:  
Suppose we had to sort an array  $A$ . A subproblem would be to sort sub-section of this array starting at index  $P$  and ending at index  $R$ , denoted as  $A[p..r]$

**Divide:** If  $q$  is half way point between  $p$  and  $r$ , we can split subarray  $A[p..r]$  into two arrays  $A[p..q]$  and  $A[q+1, r]$

**Conquer:** We try to sort both  $A[p..q]$  &  $A[q+1, r]$ . If we haven't reached base case, again divide both these subarrays and try to sort them.



**Combine:** When conquer step reaches base step and we get two sorted subarrays, we combine results by creating a sorted array  $A[p..r]$  from  $A[p..q]$  &  $A[q+1..r]$ .

Algorithm:

**mergesort** ( $A, p, r$ ):

if  $p > r$

return

$q = (p+r)/2$

**mergesort** ( $A, p, q$ )

**mergesort** ( $A, q+1, r$ )

**merge** ( $A, p, q, r$ )

**merge** ( $A, p, q, r$ ):

$n_1 = q - p + 1$

$n_2 = r - q$

Create arrays  $L[1..n_1], R[1..n_2]$

for  $i = 0$  to  $n_1 - 1$ :

$L[i] = A[p+i]$

for  $j = 0$  to  $n_2 - 1$ :

$R[j] = A[q+1+j]$

$i = 0$

$j = 0, k = p$

while  $i < n_1$  and  $j < n_2$ :

if  $L[i] \leq R[j]$ :

$A[k] = L[i]$

$i = i + 1$

else:

$A[k] = R[j]$

$j = j + 1$

$k = k + 1$

Time Complexity: Divide:  $O(1)$

Best:  $O(n \log n)$

Worst:  $O(n \log n)$

Average:  $O(n \log n)$

while  $i < n_1$ :

$A[k] = L[i]$

$i = i + 1$

$k = k + 1$

while  $j < n_2$ :

$A[k] = R[j]$

$j = j + 1$

$k = k + 1$

Per level: Merge:  $O(n)$  per level  
Total levels:  $\log(n)$



## Quick Sort

It is a divide and conquer sorting algorithm that works by selecting a pivot element, partitioning array around pivot and recursively sorting left and right subarrays.

### Divide (Partitioning):

- \* Select pivot element from array (typically last element)
- \* Rearrange array so that elements smaller than pivot are on left, elements larger than pivot are on right.
- \* The pivot is now in its final sorted position.

### Conquer (Recursive Sorting):

- \* Recursively apply Quick Sort on left subarray
- \* Recursively apply Quick Sort on right subarray.

### Combine:

- \* No explicit merge is included (unlike merge sort) since elements are already in place after partitioning.

### Algorithm:

#### Lomuto's Partition Scheme:

quicksort(A, low, high):

if low < high

    pivotIndex = partition(A, low, high)

    quicksort(A, low, pivotIndex - 1)

    quicksort(A, pivotIndex + 1, high)

partition(A, low, high):

    pivot = A[high]

    i = low - 1

    for j = low to high - 1 do:

        if A[j] ≤ pivot:

            i = i + 1

            swap A[i] and A[j]

    swap A[i + 1] and A[high]

    return i + 1



Example:  $A = [38, 27, 43, 3, 9, 82, 10]$

Pivot = 10

$[3, 9]$   $[10]$

$[38, 27, 43, 82]$

Pivot = 9

$[3]$   $[9]$   $[]$

$[38, 27, 43]$   $[82]$

Pivot = 43

$[38, 27]$

$[43]$

$[27]$

$[38]$

$[82]$

Pivot = 27

$[]$   $[27]$   $[38]$

Sorted  $[3, 9] + [10] + [27, 38, 43] + [82]$

Final Sorted Array: 3, 9, 10, 27, 38, 43, 82.

Explanation:

Step 1: Pivot = 10.

$38 > 10$  - No swap

$27 > 10$  - No swap

$43 > 10$  - No swap

$3 < 10$  - swap (3, 38)

$9 < 10$  - swap (9, 27)

Swap pivot 10 with first greater element 38.

Left - (A, 0, 1)

Right - (A, 3, 6)

Step 2: Pivot = 9.

$3 < 9$  - no swap.

Swap 9 with itself.

Left - (A, 0, 0)

Right - (A, 2, 1)



Step 3: Pivot = 43.

38 < 43 - swap 38

27 < 43 - swap 27.

Swap 43 with first greatest 43 itself.

left - (A, 3, 4)

Right - (A, 6, 6)

Step 4: Pivot = 27

38 > 27 - No swap

Swap pivot 27 with first greatest 38.

left - (A, 3, 2)

Right - (A, 4, 4)

Time Complexity:

**Best Case:  $O(n \log n)$**

If pivot divides array into 2 equal halves.  
i.e. perfectly balanced partitions. Recurrence relation  
ex:  $T(n) = 2T(n/2) + O(n)$ ; Applying Master's Theorem.

$$T(n) = O(n \log n)$$

**Worst Case:  $O(n^2)$**

If pivot always ends up being smallest or largest element resulting in highly unbalanced partition.  
Recurrence relation is:  $T(n) = T(n-1) + O(n)$ .  
where  $T(n) = O(n^2)$ .

**Average case:  $O(n \log n)$**

If pivot splits array unevenly but not in worst way, meaning partitions are still fairly balanced. Recurrence relation is:

$$T(n) = T(n/3) + T(2n/3) + O(n)$$

leads to  $T(n) = O(n \log n)$



# Dynamic Programming

It is a powerful algorithmic technique used for optimization problems where solution depends on overlapping subproblems and follows **Principle of Optimality**. Bellman introduced this technique and it is widely used in various algorithmic applications such as: Matrix Chain multiplication, 0/1 Knapsack, Shortest Path Problems and Travelling Salesman Problem.

It can be divided into multiple stages where each stage requires an optimal decision. The decision at each stage affects next stage as well as overall outcome.

## Elements of Dynamic Programming

1. Optimal Substructure: A problem has an optimal substructure if the optimal solution to problem can be constructed from optimal solution of its subpbm.

**Eg:** Shortest Path Problem. (Floyd - Warshall)

If shortest path from A to C (go through B), then path from A to B & B to C  $\rightarrow$  optimal.

$$d(A, C) = \min(d(A, B) + d(B, C))$$

where  $d(A, C)$  - shortest distance between A & C.

2. Overlapping Subproblems: A problem has overlapping subproblems if same subproblems are solved multiple times. Instead of solving them repeatedly we store their results and reuse them.

**Eg:** Fibonacci Sequence:

$$F(n) = F(n-1) + F(n-2)$$

It does lot of redundant calculations. Instead of recomputing  $F(n-2)$  &  $F(n-1)$  multiple time, store them



in an array to avoid redundant computations.

Without DP (Recursive approach):

$$T(n) = O(2^n) \text{ Exponential}$$

With DP (Memoization or Tabulation):

$$T(n) = O(n) \text{ linear.}$$

### 3. Memoization (Top-Down Approach)

- Uses recursion to break problem into smaller
- Stores results of solved subproblems to avoid recomputation.

**Eg:** Fibonacci Sequence:

```
function fibo(n, dp)
```

```
  if  $n \leq 1$  then
```

```
    return n
```

```
  end if
```

```
  if  $dp[n] \neq -1$  then
```

```
    return  $dp[n]$ 
```

```
  end if
```

```
   $dp[n] \leftarrow \text{fibo}(n-1, dp) + \text{fibo}(n-2, dp)$ 
```

```
  return  $dp[n]$ 
```

We store computed values in array  $dp[]$ .

This reduces time complexity from  $O(2^n)$  to  $O(n)$ .

### 4. Tabulation (Bottom-Up Approach)

- Iteratively build solution from smallest subproblem to find solution.

- Uses array (table) to store computed results iteratively.

**Eg:** Fibonacci Sequence:

```
fn. fibo(n)
```

```
  create array dp of size  $(n+1)$ 
```

```
   $dp[0] = 0$ 
```

```
   $dp[1] = 1$ 
```



for  $i$  from 2 to  $n$

$$dp[i] = dp[i-1] + dp[i-2]$$

return  $dp[n]$ .

No recursion is used and results are built iteratively.  
More space-efficient if implemented with constant space:  $O(1)$ .

### Steps to solve Dynamic Programming:

1. Characterize structure of an optimal solution: Identify how problem can be broken into subproblems and ensure optimal substructure.
2. Define value of an optimal solution recursively: Formulate recurrence relation that represents problem.
3. Compute optimal solution iteratively (Bottom up): Use memoization (top down) or tabulation (bottom-up) to compute results efficiently.
4. Construct optimal solution using stored/computed results: Rebuild solution using stored table.

### Applications:

1. Matrix Chain Multiplication
2. Optimal Binary Search Trees
3. 0/1 Knapsack Problem
4. Multistage Graph
5. Travelling Salesman Problem
6. Reliability Design

### Matrix Chain Multiplication.

**Input:**  $n$  matrices  $A_1, A_2, A_3, \dots, A_n$  of dimensions  $p_1 \times p_2, p_2 \times p_3, \dots, p_n \times p_{n+1}$  respectively.

**Goal:** To compute matrix product  $A_1 A_2 \dots A_n$



**Problem:** In what order should  $A_1, A_2, \dots, A_n$  be multiplied so that it would take min. no. of computations to derive the product.

Let  $A$  &  $B$  be two matrices of dimensions  $p \times q, q \times r$ . Then  $C = AB$ .  $C$  is of dimension  $p \times r$ . Thus  $C_{ij}$  takes  $q$  scalar multiplications and  $q-1$  scalar additions.

**Eg:**

Let  $A_1 - 5 \times 4$

$A_2 - 4 \times 6$

$A_3 - 6 \times 2$

$(A_1 A_2) A_3$  takes  $(5 \times 4 \times 6) + (5 \times 6 \times 2) = 180$

$A_1 (A_2 A_3)$  takes  $(5 \times 4 \times 2) + (4 \times 6 \times 2) = 88$ . This is much cheaper to compute than  $(A_1 A_2) \cdot A_3$

**Step 1:** Let  $M_{ij}$  - cost of multiplying  $A_i \dots A_j$  where cost is measured in no. of multiplications.  $\therefore$

$M(i, i) = 0$  for all  $i$ .

$M(1, n) \rightarrow$  required solution.

**Step 2:** Sequence of decision be build using principle of optimality.

Let  $T \rightarrow$  tree corresponding to optimal way of multiplying  $A_i \dots A_j$

$T$  has left subtree  $L$  & right subtree  $R$ .  $L$  corresponds to multiplying  $A_i \dots A_k$  and  $R$  to  $A_{k+1} \dots A_j$  for  $i \leq k \leq j-1$ . Thus we get optimal sub-chain of matrices and then multiplication is performed. This proves matrix chain multiplication.

**Step 3:** Apply foll. formula for computing each sequence

$$M_{ij} = \min_k [M_{ik} + M_{k+1, j} + P_i P_{k+1} P_{j+1}] \quad ; i \leq k \leq j-1$$



Example 1:

Consider  $A_1 = 5 \times 4$ ,  $A_2 = 4 \times 6$ ,  $A_3 = 6 \times 2$ ,  $A_4 = 2 \times 7$   
 $P_1 = 5$ ,  $P_2 = 4$ ,  $P_3 = 6$ ,  $P_4 = 2$ ,  $P_5 = 7$ .

|   |                         |                         |                        |              |   |
|---|-------------------------|-------------------------|------------------------|--------------|---|
|   |                         | 1                       | 2                      | 3            | 4 |
| 0 | $M_{11} = 0$            | $M_{22} = 0$            | $M_{33} = 0$           | $M_{44} = 0$ |   |
| 1 | $M_{12} = 120$<br>$k=1$ | $M_{23} = 48$<br>$k=2$  | $M_{34} = 84$<br>$k=3$ |              |   |
| 2 | $M_{13} = 88$<br>$k=1$  | $M_{24} = 104$<br>$k=3$ |                        |              |   |
| 3 | $M_{14} = 158$<br>$k=3$ |                         |                        |              |   |

Computation of  $M_{ij}$ .

Let  $i=1, j=2, k=1$ .

$$M_{ij} \Rightarrow M_{12} = M_{11} + M_{22} + P_1 P_2 P_3 = 0 + 0 + 5 \times 4 \times 6 = 120$$

$i=2, j=3, k=2$

$$M_{23} = M_{22} + M_{33} + P_2 P_3 P_4 = 0 + 0 + 4 \times 6 \times 2 = 48$$

$i=3, j=4, k=3$

$$M_{34} = M_{33} + M_{44} + P_3 P_4 P_5 = 0 + 0 + 6 \times 2 \times 7 = 84$$

$i=1, j=3, k=1$  or  $k=2$

$$k=1 \rightarrow M_{13} = M_{11} + M_{23} + P_1 P_2 P_4 = 0 + 48 + 5 \times 4 \times 2 = 88 \text{ (min)}$$

$$k=2 \rightarrow M_{13} = M_{12} + M_{33} + P_1 P_3 P_4 = 120 + 0 + 60 = 180$$

As we get min. value of  $M_{13}$  when  $k=1$ , we set

$$k=1 \text{ \& } M_{13} = 88.$$

$i=2, j=4, k=2$  or  $k=3$

$$k=2 \rightarrow M_{24} = M_{22} + M_{34} + P_2 P_3 P_5 = 0 + 84 + 4 \times 6 \times 7 = 252$$

$$k=3 \rightarrow M_{24} = M_{23} + M_{44} + P_2 P_4 P_5 = 48 + 0 + 4 \times 2 \times 7 = 104 \text{ (min)}$$

$i=1, j=4, k=1$  or  $2$  or  $3$ .

$$k=1 \rightarrow M_{14} = M_{11} + M_{24} + P_1 P_2 P_5 = 0 + 104 + 5 \times 4 \times 7 = 244$$

$$k=2 \rightarrow M_{14} = M_{12} + M_{34} + P_1 P_3 P_5 = 120 + 84 + 5 \times 6 \times 7 = 414$$

$$k=3 \rightarrow M_{14} = M_{13} + M_{44} + P_1 P_4 P_5 = 88 + 0 + 5 \times 2 \times 7 = 158 \text{ (min)}$$



Thus we get Optimum cost as 158. To seek the optimum sequence for this cost we will trace algorithm Mul. for this cost 158 which is optimal.

∴ Optimal Sequence is  $A_1 \times (A_2 \times A_3) \times A_4$ .

For instance

$$A_1 = 5 \times 4$$

$$A_2 = 4 \times 6$$

$$A_3 = 6 \times 2$$

$$A_4 = 2 \times 7$$

$$A_1 \times (A_2 \times A_3) \times A_4 = 158.$$

Algorithm:

1. Matrix chain Multiplication:

fn matrixchain(array  $P[1 \dots n]$ )

for  $i := 1$  to  $n$  do

$M[i, i] := 0$  // Initialisation

for  $len = 2$  to  $n$

for  $i = 1$  to  $(n - len + 1)$

$j = i + len - 1$

$M[i, j] = \infty$

for  $k = i$  to  $j - 1$

$q = M[i, k] + M[k + 1, j] + P[i] * P[k + 1] * P[j + 1]$

if  $q < (M[i, j])$

$M[i, j] = q$

$S[i, j] = k$

// Split matrix

return  $M[1, n]$  // optimum value.

2. Compute Optimum Sequence:

fn mul( $i, j$ )

//  $P = A[i] \dots A[k]$

if  $(i = j)$  then

//  $Q = A[k + 1] \dots A[j]$

return  $A[i]$

else

$M_{14}$   
 $k=3$   
 $A_1, A_2, A_3, A_4$

$M_{13}$   
 $k=1$   
 $A_1, A_2, A_3$

$M_{44}$   
 $A_4$

$M_{11}$   
 $A_1$

$M_{23}$   
 $k=2$   
 $A_2, A_3$



$$k = S[i, j]$$

$$P = \text{Mul}(i, k)$$

$$Q = \text{Mul}(k+1, j)$$

$$\text{return } P * Q$$

In the algorithm `matrix_chain_mul` first for loop takes  $O(n)$  for initialization of  $M[i, j]$ . If  $M[i, j]$  not computed previously then using three nested for loops each  $M[i, j]$  called many times. This leads to  $O(n^3)$  time complexity.

Example 2: Find optimal parenthesization of matrix chain product whose sequence of dimension is  $(5, 10, 3, 12, 5, 50, 6)$  using DP.

Soln:

$$P_0 = 5, P_1 = 10, P_2 = 3, P_3 = 12, P_4 = 5, P_5 = 50, P_6 = 6$$

$$1 \leq i \leq 5$$

|   | $A_1$ | $A_2$ | $A_3$ | $A_4$ | $A_5$ | $A_6$ |
|---|-------|-------|-------|-------|-------|-------|
| 6 | 0     | 0     | 0     | 0     | 0     | 0     |
| 5 | 150   | 360   | 180   | 3000  | 5000  |       |
| 4 | 330   | 330   | 930   | 1860  |       |       |
| 3 | 405   | 2430  | 1770  |       |       |       |
| 2 | 1655  | 1950  |       |       |       |       |
| 1 | 2010  |       |       |       |       |       |

Matrix Table

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 |   |   |   |   |   |
| 2 | 2 |   |   |   |   |
| 3 | 2 | 2 |   |   |   |
| 4 | 4 | 2 |   |   |   |
| 5 | 2 |   |   |   |   |

S-Table

Optimal Parenthesization will be:

$$(A_1 * A_2) * ((A_3 * A_4) * (A_5 * A_6))$$

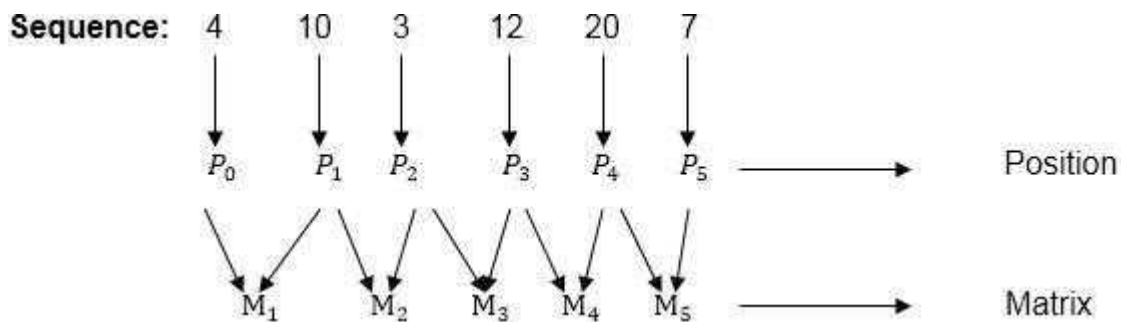


### Example of Matrix Chain Multiplication

Example: We are given the sequence {4, 10, 3, 12, 20, and 7}. The matrices have size 4 x 10, 10 x 3, 3 x 12, 12 x 20, 20 x 7. We need to compute  $M[i, j]$ ,  $0 \leq i, j \leq 5$ . We know  $M[i, i] = 0$  for all  $i$ .

| 1 | 2 | 3 | 4 | 5 |   |
|---|---|---|---|---|---|
| 0 |   |   |   |   | 1 |
|   | 0 |   |   |   | 2 |
|   |   | 0 |   |   | 3 |
|   |   |   | 0 |   | 4 |
|   |   |   |   | 0 | 5 |

Let us proceed with working away from the diagonal. We compute the optimal solution for the product of 2 matrices.



In Dynamic Programming, initialization of every method done by '0'. So we initialize it by '0'. It will sort out diagonally.

We have to sort out all the combination but the minimum output combination is taken into consideration.

Calculation of Product of 2 matrices:

$$1. m(1, 2) = m_1 \times m_2$$

$$= 4 \times 10 \times 10 \times 3$$

$$= 4 \times 10 \times 3 = 120$$

$$2. m(2, 3) = m_2 \times m_3$$



$$= 10 \times 3 \times 3 \times 12$$

$$= 10 \times 3 \times 12 = 360$$

$$3. m(3, 4) = m_3 \times m_4$$

$$= 3 \times 12 \times 12 \times 20$$

$$= 3 \times 12 \times 20 = 720$$

$$4. m(4,5) = m_4 \times m_5$$

$$= 12 \times 20 \times 20 \times 7$$

$$= 12 \times 20 \times 7 = 1680$$

| 1 | 2   | 3   | 4   | 5    |   |
|---|-----|-----|-----|------|---|
| 0 | 120 |     |     |      | 1 |
|   | 0   | 360 |     |      | 2 |
|   |     | 0   | 720 |      | 3 |
|   |     |     | 0   | 1680 | 4 |
|   |     |     |     | 0    | 5 |

We initialize the diagonal element with equal i,j value with '0'.

After that second diagonal is sorted out and we get all the values corresponded to it Now the third diagonal will be solved out in the same way.

Now product of 3 matrices:

$$M[1, 3] = M_1 M_2 M_3$$

There are two cases by which we can solve this multiplication:  $(M_1 \times M_2) + M_3$ ,  $M_1 + (M_2 \times M_3)$

After solving both cases we choose the case in which minimum output is there.

$$M[1, 3] = \min \left\{ \begin{array}{l} M[1,2] + M[3,3] + p_0 p_2 p_3 = 120 + 0 + 4.3.12 = 264 \\ M[1,1] + M[2,3] + p_0 p_1 p_3 = 0 + 360 + 4.10.12 = 840 \end{array} \right\}$$

$$M[1, 3] = 264$$

As Comparing both output 264 is minimum in both cases so we insert 264 in table and  $(M_1 \times M_2) + M_3$  this combination is chosen for the output making.

$$M[2, 4] = M_2 M_3 M_4$$

There are two cases by which we can solve this multiplication:  $(M_2 \times M_3) + M_4$ ,  $M_2 + (M_3 \times M_4)$



After solving both cases we choose the case in which minimum output is there.

$$M[2, 4] = \min \begin{cases} M[2,3] + M[4,4] + p_1 p_3 p_4 = 360 + 0 + 10 \cdot 12 \cdot 20 = 2760 \\ M[2,2] + M[3,4] + p_1 p_2 p_4 = 0 + 720 + 10 \cdot 3 \cdot 20 = 1320 \end{cases}$$

$$M[2, 4] = 1320$$

As Comparing both output 1320 is minimum in both cases so we insert 1320 in table and  $M_2 + (M_3 \times M_4)$  this combination is chosen for the output making.

$$M[3, 5] = M_3 M_4 M_5$$

There are two cases by which we can solve this multiplication:  $(M_3 \times M_4) + M_5$ ,  $M_3 + (M_4 \times M_5)$

After solving both cases we choose the case in which minimum output is there.

$$M[3, 5] = \min \begin{cases} M[3,4] + M[5,5] + p_2 p_4 p_5 = 720 + 0 + 3 \cdot 20 \cdot 7 = 1140 \\ M[3,3] + M[4,5] + p_2 p_3 p_5 = 0 + 1680 + 3 \cdot 12 \cdot 7 = 1932 \end{cases}$$

$$M[3, 5] = 1140$$

As Comparing both output 1140 is minimum in both cases so we insert 1140 in table and  $(M_3 \times M_4) + M_5$  this combination is chosen for the output making.

| 1 | 2   | 3   | 4   | 5    |   |
|---|-----|-----|-----|------|---|
| 0 | 120 |     |     |      | 1 |
|   | 0   | 360 |     |      | 2 |
|   |     | 0   | 720 |      | 3 |
|   |     |     | 0   | 1680 | 4 |
|   |     |     |     | 0    | 5 |

→

| 1 | 2   | 3   | 4    | 5    |   |
|---|-----|-----|------|------|---|
| 0 | 120 | 264 |      |      | 1 |
|   | 0   | 360 | 1320 |      | 2 |
|   |     | 0   | 720  | 1140 | 3 |
|   |     |     | 0    | 1680 | 4 |
|   |     |     |      | 0    | 5 |

Now Product of 4 matrices:

$$M[1, 4] = M_1 M_2 M_3 M_4$$

There are three cases by which we can solve this multiplication:

$$(M_1 \times M_2 \times M_3) M_4$$

$$M_1 \times (M_2 \times M_3 \times M_4) \quad 3. \quad (M_1 \times M_2) \times (M_3 \times M_4)$$

After solving these cases we choose the case in which minimum output is there

$$M[1, 4] = \min \begin{cases} M[1,3] + M[4,4] + p_0 p_3 p_4 = 264 + 0 + 4 \cdot 12 \cdot 20 = 1224 \\ M[1,2] + M[3,4] + p_0 p_2 p_4 = 120 + 720 + 4 \cdot 3 \cdot 20 = 1080 \\ M[1,1] + M[2,4] + p_0 p_1 p_4 = 0 + 1320 + 4 \cdot 10 \cdot 20 = 2120 \end{cases}$$

$$M[1, 4] = 1080$$



As comparing the output of different cases then '1080' is minimum output, so we insert 1080 in the table and (M1 x M2) x (M3 x M4) combination is taken out in output making,

$$M[2, 5] = M2 \ M3 \ M4 \ M5$$

There are three cases by which we can solve this multiplication:

$$(M2 \times M3 \times M4) \times M5$$

$$M2 \times (M3 \times M4 \times M5)$$

$$3. (M2 \times M3) \times (M4 \times M5)$$

After solving these cases we choose the case in which minimum output is there

$$M[2, 5] = \min \begin{cases} M[2,4] + M[5,5] + p_1 p_4 p_5 = 1320 + 0 + 10.20.7 = 2720 \\ M[2,3] + M[4,5] + p_1 p_3 p_5 = 360 + 1680 + 10.12.7 = 2880 \\ M[2,2] + M[3,5] + p_1 p_2 p_5 = 0 + 1140 + 10.3.7 = 1350 \end{cases}$$

$$M[2, 5] = 1350$$

As comparing the output of different cases then '1350' is minimum output, so we insert 1350 in the table and M2 x( M3 x M4xM5)combination is taken out in output making.

| 1 | 2   | 3   | 4    | 5    |   | 1 | 2   | 3   | 4    | 5    |   |
|---|-----|-----|------|------|---|---|-----|-----|------|------|---|
| 0 | 120 | 264 |      |      | 1 | 0 | 120 | 264 | 1080 |      | 1 |
|   | 0   | 360 | 1320 |      | 2 |   | 0   | 360 | 1320 | 1350 | 2 |
|   |     | 0   | 720  | 1140 | 3 |   |     | 0   | 720  | 1140 | 3 |
|   |     |     | 0    | 1680 | 4 |   |     |     | 0    | 1680 | 4 |
|   |     |     |      | 0    | 5 |   |     |     |      | 0    | 5 |

Now Product of 5 matrices:

$$M[1, 5] = M1 \ M2 \ M3 \ M4 \ M5$$

There are five cases by which we can solve this multiplication:

$$(M1 \times M2 \times M3 \times M4) \times M5$$

$$M1 \times (M2 \times M3 \times M4 \times M5)$$

$$(M1 \times M2 \times M3) \times M4 \times M5$$

$$M1 \times M2 \times (M3 \times M4 \times M5)$$

After solving these cases we choose the case in which minimum output is there



$$M[1, 5] = \min \begin{cases} M[1,4] + M[5,5] + p_0 p_4 p_5 = 1080 + 0 + 4.20.7 = 1544 \\ M[1,3] + M[4,5] + p_0 p_3 p_5 = 264 + 1680 + 4.12.7 = 2016 \\ M[1,2] + M[3,5] + p_0 p_2 p_5 = 120 + 1140 + 4.3.7 = 1344 \\ M[1,1] + M[2,5] + p_0 p_1 p_5 = 0 + 1350 + 4.10.7 = 1630 \end{cases}$$

$M[1, 5] = 1344$

As comparing the output of different cases then '1344' is minimum output, so we insert 1344 in the table and  $M_1 \times M_2 \times (M_3 \times M_4 \times M_5)$  combination is taken out in output making.

Final Output is:

| 1 | 2   | 3   | 4    | 5    |   |
|---|-----|-----|------|------|---|
| 0 | 120 | 264 | 1080 |      | 1 |
|   | 0   | 360 | 1320 | 1350 | 2 |
|   |     | 0   | 720  | 1140 | 3 |
|   |     |     | 0    | 1680 | 4 |
|   |     |     |      | 0    | 5 |

→

| 1 | 2   | 3   | 4    | 5    |   |
|---|-----|-----|------|------|---|
| 0 | 120 | 264 | 1080 | 1344 | 1 |
|   | 0   | 360 | 1320 | 1350 | 2 |
|   |     | 0   | 720  | 1140 | 3 |
|   |     |     | 0    | 1680 | 4 |
|   |     |     |      | 0    | 5 |

So we can get the optimal solution of matrices multiplication....

## Multi Stage Graph

Multistage Graph problem is defined as follow:

Multistage graph  $G = (V, E, W)$  is a weighted directed graph in which vertices are partitioned into  $k \geq 2$  disjoint sub sets  $V = \{V_1, V_2, \dots, V_k\}$  such that if edge  $(u, v)$  is present in  $E$  then  $u \in V_i$  and  $v \in V_{i+1}$ ,  $1 \leq i \leq k$ . The goal of multistage graph problem is to find minimum cost path from source to destination vertex.

The input to the algorithm is a k-stage graph, n vertices are indexed in increasing order of stages.

The algorithm operates in the backward direction, i.e. it starts from the last vertex of the graph and proceeds in a backward direction to find minimum cost path.

Minimum cost of vertex  $j \in V_i$  from vertex  $r \in V_{i+1}$  is defined as,  $\text{Cost}[j] = \min\{ c[j, r] + \text{cost}[r] \}$

where,  $c[j, r]$  is the weight of edge  $\langle j, r \rangle$  and  $\text{cost}[r]$  is the cost of moving from end vertex to vertex  $r$ .

**Algorithm for the multistage graph is described below :**

Algorithm for Multistage Graph Algorithm MULTI\_STAGE( $G, k, n, p$ )

// Description: Solve multi-stage problem using dynamic programming

// Input:



k: Number of stages in graph  $G = (V, E)$   $c[i, j]$ : Cost of edge  $(i, j)$

// Output:  $p[1:k]$ : Minimum cost path  $\text{cost}[n] \leftarrow 0$

for  $j \leftarrow n - 1$  to 1 do

// Let  $r$  be a vertex such that  $(j, r) \in E$  and  $c[j, r] + \text{cost}[r]$  is minimum  $\text{cost}[j] \leftarrow c[j, r] + \text{cost}[r]$

$\pi[j] \leftarrow r$

end

// Find minimum cost path  $p[1] \leftarrow 1$

$p[k] \leftarrow n$

for  $j \leftarrow 2$  to  $k - 1$  do

$p[j] \leftarrow \pi[p[j - 1]]$

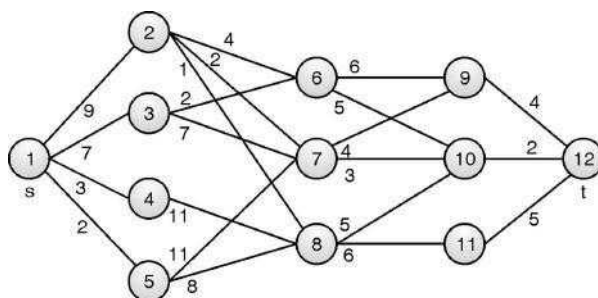
end

### Complexity Analysis of Multistage Graph

If graph  $G$  has  $|E|$  edges, then cost computation time would be  $O(n + |E|)$ . The complexity of tracing the minimum cost path would be  $O(k)$ ,  $k < n$ . Thus total time complexity of multistage graph using dynamic programming would be  $O(n + |E|)$ .

### Example

Example: Find minimum path cost between vertex  $s$  and  $t$  for following multistage graph using dynamic programming.



### Solution:

Solution to multistage graph using dynamic programming is constructed as,  $\text{Cost}[j] = \min\{c[j, r] + \text{cost}[r]\}$

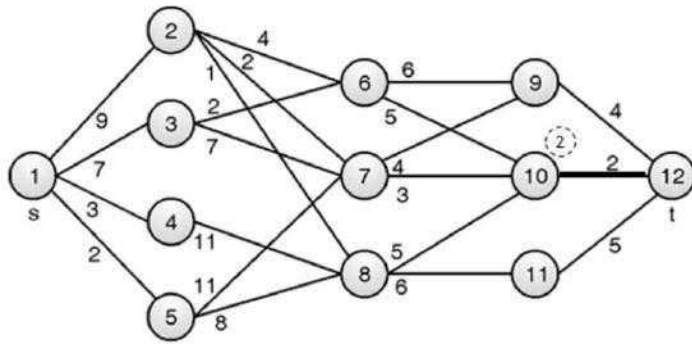
Here, number of stages  $k = 5$ , number of vertices  $n = 12$ , source  $s = 1$  and target  $t = 12$  Initialization:

$\text{Cost}[n] = 0 \Rightarrow \text{Cost}[12] = 0$ .  $p[1] = s \Rightarrow p[1] = 1$

$p[k] = t \Rightarrow p[5] = 12$ .  $r = t = 12$ .

Stage 4:





Stage 3:

Vertex 6 is connected to vertices 9 and 10:

$$\text{Cost}[6] = \min\{ c[6, 10] + \text{Cost}[10], c[6, 9] + \text{Cost}[9] \}$$

$$= \min\{5 + 2, 6 + 4\} = \min\{7, 10\} = 7$$

$$p[6] = 10$$

Vertex 7 is connected to vertices 9 and 10:

$$\text{Cost}[7] = \min\{ c[7, 10] + \text{Cost}[10], c[7, 9] + \text{Cost}[9] \}$$

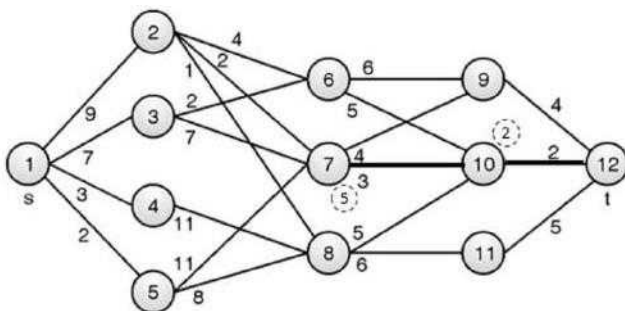
$$= \min\{3 + 2, 4 + 4\} = \min\{5, 8\} = 5$$

$$p[7] = 10$$

Vertex 8 is connected to vertex 10 and 11:

$$\text{Cost}[8] = \min\{ c[8, 11] + \text{Cost}[11], c[8, 10] + \text{Cost}[10] \}$$

$$= \min\{6 + 5, 5 + 2\} = \min\{11, 7\} = 7 \quad p[8] = 10$$



Stage 2:

Vertex 2 is connected to vertices 6, 7 and 8:

$$\text{Cost}[2] = \min\{ c[2, 6] + \text{Cost}[6], c[2, 7] + \text{Cost}[7], c[2, 8] + \text{Cost}[8] \}$$

$$= \min\{4 + 7, 2 + 5, 1 + 7\} = \min\{11, 7, 8\} = 7$$



$$p[2] = 7$$

Vertex 3 is connected to vertices 6 and 7:

$$\text{Cost}[3] = \min\{ c[3, 6] + \text{Cost}[6], c[3, 7] + \text{Cost}[7] \}$$

$$= \min\{2 + 7, 7 + 5\} = \min\{9, 12\} = 9$$

$$p[3] = 6$$

Vertex 4 is connected to vertex 8:

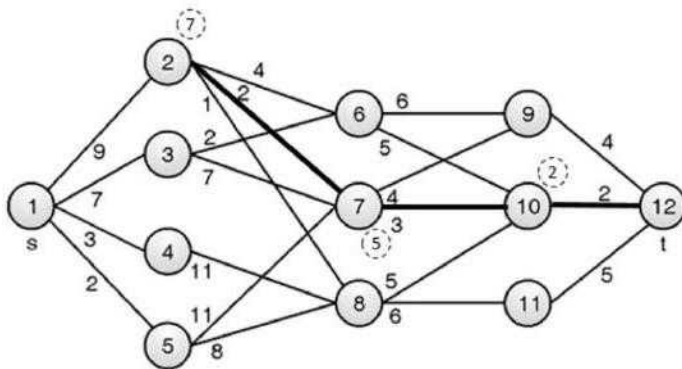
$$\text{Cost}[4] = c[4, 8] + \text{Cost}[8] = 11 + 7 = 18$$

$$p[4] = 8$$

Vertex 5 is connected to vertices 7 and 8:

$$\text{Cost}[5] = \min\{ c[5, 7] + \text{Cost}[7], c[5, 8] + \text{Cost}[8] \}$$

$$= \min\{11 + 5, 8 + 7\} = \min\{16, 15\} = 15 \quad p[5] = 8$$



Stage 1:

Vertex 1 is connected to vertices 2, 3, 4 and 5:

$$\text{Cost}[1] = \min\{ c[1, 2] + \text{Cost}[2], c[1, 3] + \text{Cost}[3], c[1, 4] + \text{Cost}[4], c[1, 5] + \text{Cost}[5] \}$$

$$= \min\{ 9 + 7, 7 + 9, 3 + 18, 2 + 15 \}$$

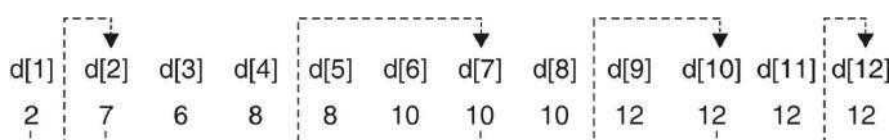
$$= \min\{ 16, 16, 21, 17 \} = 16 \quad p[1] = 2$$

Trace the solution:

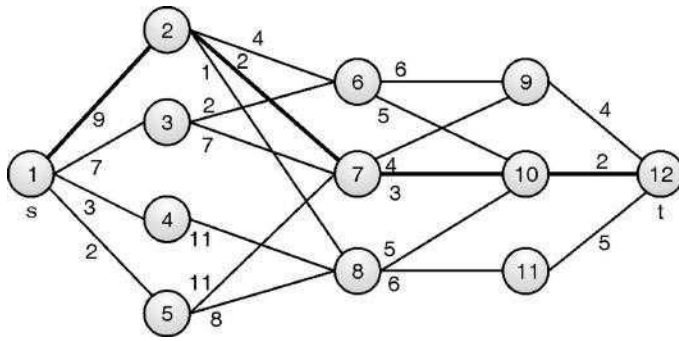
$$p[1] = 2$$

$$p[2] = 7$$

$$p[7] = 10$$







$p[10] = 12$

Minimum cost path is : 1 – 2 – 7 – 10 – 12

Cost of the path is :  $9 + 2 + 3 + 2 = 16$

### Optimal Binary Search Tree

Optimal Binary Search Tree extends the concept of Binary search tree. Binary Search Tree (BST) is a nonlinear data structure which is used in many scientific applications for reducing the search time. In BST, left child is smaller than root and right child is greater than root. This arrangement simplifies the search procedure.

Optimal Binary Search Tree (OBST) is very useful in dictionary search. The probability of searching is different for different words. OBST has great application in translation.

If we translate the book from English to German, equivalent words are searched from English to German dictionary and replaced in translation. Words are searched same as in binary search tree order.

Binary search tree simply arranges the words in lexicographical order. Words like ‘the’, ‘is’, ‘there’ are very frequent words, whereas words like ‘xylophone’, ‘anthropology’ etc. appears rarely.

It is not a wise idea to keep less frequent words near root in binary search tree. Instead of storing words in binary search tree in lexicographical order, we shall arrange them according to their probabilities. This arrangement facilitates few searches for frequent words as they would be near the root. Such tree is called Optimal Binary Search Tree.

Consider the sequence of  $n$  keys  $K = \langle k_1, k_2, k_3, \dots, k_n \rangle$  of distinct probability in sorted order such that

$k_1 < k_2 < \dots < k_n$ . Words between each pair of key lead to unsuccessful search, so for  $n$  keys, binary search tree contains  $n + 1$  dummy keys  $d_i$ , representing unsuccessful searches.

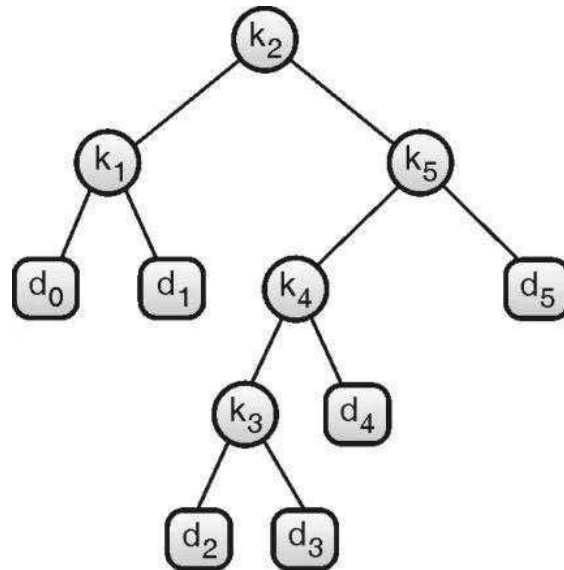
Two different representation of BST with same five keys  $\{k_1, k_2, k_3, k_4, k_5\}$  probability is shown in following figure

With  $n$  nodes, there exist  $(2n)! / ((n + 1)! * n!)$  different binary search trees. An exhaustive search for optimal binary search tree leads to huge amount of time.

The goal is to construct a tree which minimizes the total search cost. Such tree is called optimal binary search tree. OBST does not claim minimum height. It is also not necessary that parent of sub tree has higher priority than its child.

Dynamic programming can help us to find such optima tree.





Binary search trees with 5 keys

Mathematical formulation

We formulate the OBST with following observations

Any sub tree in OBST contains keys in sorted order  $k_i \dots k_j$ , where  $1 \leq i \leq j \leq n$ .

Sub tree containing keys  $k_i \dots k_j$  has leaves with dummy keys  $d_{i-1} \dots d_j$ .

Suppose  $k_r$  is the root of sub tree containing keys  $k_i \dots k_j$ . So, left sub tree of root  $k_r$  contains keys

$k_i \dots k_{r-1}$  and right sub tree contain keys  $k_{r+1}$  to  $k_j$ . Recursively, optimal sub trees are constructed from the left and right sub trees of  $k_r$ .

Let  $e[i, j]$  represents the expected cost of searching OBST. With  $n$  keys, our aim is to find and minimize  $e[1, n]$ .

Base case occurs when  $j = i - 1$ , because we just have the dummy key  $d_{i-1}$  for this case. Expected search cost for this case would be  $e[i, j] = e[i, i - 1] = q_{i-1}$ .

For the case  $j \geq i$ , we have to select any key  $k_r$  from  $k_i \dots k_j$  as a root of the tree.

With  $k_r$  as a root key and sub tree  $k_i \dots k_j$ , sum of probability is defined as

$$w(i, j) = \sum_{m=i}^j p_m + \sum_{m=i-1}^j q_m$$

(Actual key starts at index 1 and dummy key starts at index 0)

Thus, a recursive formula for forming the OBST is stated below :



$$e[i, j] = \begin{cases} q_{i-1} & \text{if } j = i - 1 \\ \min_{i \leq r \leq j} \{e[i, r-1] + e[r+1, j] + w(i, j)\} & \text{if } i \leq j \end{cases}$$

$e[i, j]$  gives the expected cost in the optimal binary search tree.

#### Algorithm for Optimal Binary Search Tree

The algorithm for optimal binary search tree is specified below :

Algorithm OBST(p, q, n)

//  $e[1 \dots n+1, 0 \dots n]$  : Optimal sub tree

//  $w[1 \dots n+1, 0 \dots n]$  : Sum of probability

//  $root[1 \dots n, 1 \dots n]$  : Used to construct OBST

for  $i \leftarrow 1$  to  $n + 1$  do

$e[i, i - 1] \leftarrow q_{i-1}$

$w[i, i - 1] \leftarrow q_{i-1}$

end

for  $m \leftarrow 1$  to  $n$  do

for  $i \leftarrow 1$  to  $n - m + 1$  do

$j \leftarrow i + m - 1$   $e[i, j] \leftarrow \infty$

$w[i, j] \leftarrow w[i, j - 1] + p_j + q_j$

for  $r \leftarrow i$  to  $j$  do

$t \leftarrow e[i, r - 1] + e[r + 1, j] + w[i, j]$

if  $t < e[i, j]$  then  $e[i, j] \leftarrow t$   $root[i, j] \leftarrow r$

end end

end end

return (e, root)

#### Complexity Analysis of Optimal Binary Search Tree

It is very simple to derive the complexity of this approach from the above algorithm. It uses three nested loops. Statements in the innermost loop run in  $O(1)$  time. The running time of the algorithm is computed as



$$\begin{aligned}
 T(n) &= \sum_{m=1}^n \sum_{i=1}^{n-m+1} \sum_{j=i}^{n-l+1} \Theta(1) \\
 &= \sum_{m=1}^n \sum_{i=1}^{n-m+1} n = \sum_{m=1}^n n^2 \\
 &= \Theta(n^3)
 \end{aligned}$$

Thus, the OBST algorithm runs in cubic time

### Example

Problem: Let  $p(1:3) = (0.5, 0.1, 0.05)$   $q(0:3) = (0.15, 0.1, 0.05, 0.05)$  Compute and construct OBST for above values using Dynamic approach. Solution:

Here, given that

|    |      |     |      |      |
|----|------|-----|------|------|
| i  | 0    | 1   | 2    | 3    |
| pi |      | 0.5 | 0.1  | 0.05 |
| qi | 0.15 | 0.1 | 0.05 | 0.05 |

Recursive formula to solve OBST problem is

$$e[i, j] = \begin{cases} q_{i-1} & \text{if } j = i - 1 \\ \min_{i \leq r \leq j} \{e[i, r-1] + e[r+1, j] + w(i, j)\} & \text{if } i \leq j \end{cases}$$

Where,

$$w(i, j) = \sum_{m=i}^j p_m + \sum_{m=i-1}^j q_m$$

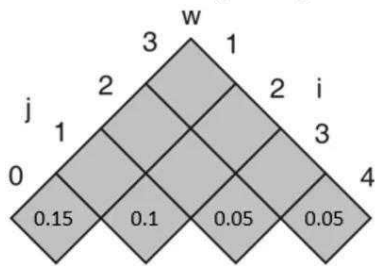
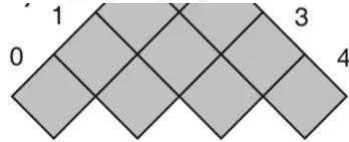
Initially,



$$v \quad w[2, 1] = \sum_{m=2}^1 p_m + \sum_{m=1}^1 q_m = q_1 = 0.1 = 0.15$$

$$w[3, 2] = \sum_{m=3}^2 p_m + \sum_{m=2}^2 q_m = q_2 = 0.05$$

$$w[4, 3] = \sum_{m=4}^3 p_m + \sum_{m=3}^3 q_m = q_3 = 0.05$$



$$w[1, 1] = \sum_{m=1}^1 p_m + \sum_{m=0}^1 q_m = p_1 + q_0 + q_1$$

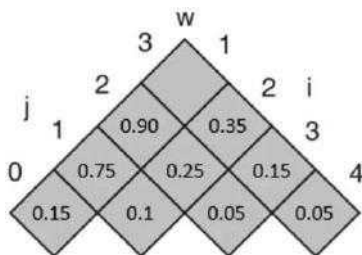
$$= 0.5 + 0.25 = 0.75$$

$$w[2, 2] = \sum_{m=2}^2 p_m + \sum_{m=1}^2 q_m = p_2 + q_1 + q_2$$

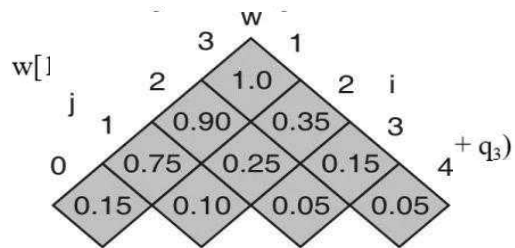
$$= 0.1 + 0.15 = 0.25$$

$$w[3, 3] = \sum_{m=3}^3 p_m + \sum_{m=2}^3 q_m = p_3 + q_2 + q_3$$

$$= \underbrace{0.05} + \underbrace{0.10} = \underbrace{0.15}$$

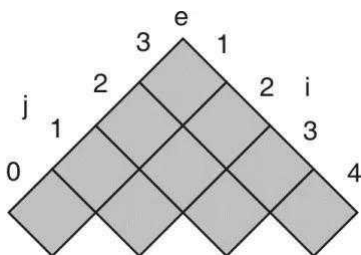






Now, we will compute  $e[i, j]$

Initially,

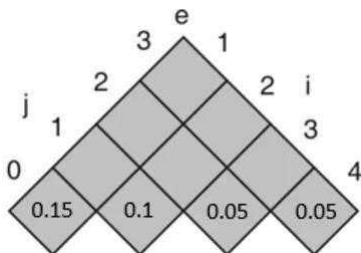


$$e[1, 0] = q_0 = 0.15 (\because j = i - 1)$$

$$e[2, 1] = q_1 = 0.1 (\because j = i - 1)$$

$$e[3, 2] = q_2 = 0.05 (\because j = i - 1)$$

$$e[4, 3] = q_3 = 0.05 (\because j = i - 1)$$



$$e[1, 1] = \min \{ e[1, 0] + e[2, 1] + w(1, 1) \}$$

$$= \min \{ 0.15 + 0.1 + 0.75 \} = 1.0$$

$$e[2, 2] = \min \{ e[2, 1] + e[3, 2] + w(2, 2) \}$$

$$= \min \{ 0.1 + 0.05 + 0.25 \} = 0.4$$



$$e[3, 3] = \min \{ e[3, 2] + e[4, 3] + w(3, 3) \}$$

$$= \min \{ 0.05 + 0.05 + 0.15 \} = 0.25$$

$$e[1, 2] = \min \left\{ \begin{array}{l} e[1, 0] + e[2, 2] + w(1, 2) \\ e[1, 1] + e[3, 2] + w(1, 2) \end{array} \right\}$$

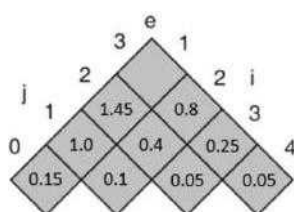
$$= \min \left\{ \begin{array}{l} 0.15 + 0.4 + 0.90 \\ 1.0 + 0.05 + 0.90 \end{array} \right\}$$

$$= \min \left\{ \begin{array}{l} 1.45 \\ 1.95 \end{array} \right\} = 1.45$$

$$e[2, 3] = \min \left\{ \begin{array}{l} e[2, 1] + e[3, 3] + w(2, 3) \\ e[2, 2] + e[4, 3] + w(2, 3) \end{array} \right\}$$

$$= \min \left\{ \begin{array}{l} 0.1 + 0.25 + 0.35 \\ 0.4 + 0.05 + 0.35 \end{array} \right\}$$

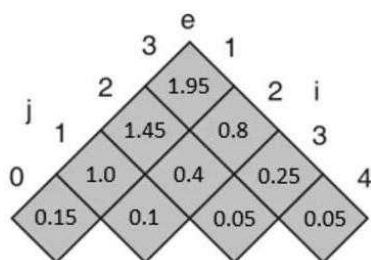
$$= \min \left\{ \begin{array}{l} 0.90 \\ 0.80 \end{array} \right\} = 0.8$$



$$e[1, 3] = \min \left\{ \begin{array}{l} e[1, 0] + e[2, 3] + w(1, 3) \\ e[1, 1] + e[3, 3] + w(1, 3) \\ e[1, 2] + e[4, 3] + w(1, 3) \end{array} \right\}$$

$$= \min \left\{ \begin{array}{l} 0.15 + 0.80 + 1.0 \\ 1.0 + 0.25 + 1.0 \\ 1.45 + 0.05 + 1.0 \end{array} \right\}$$

$$= \min \left\{ \begin{array}{l} 1.95 \\ 2.25 \\ 2.5 \end{array} \right\} = 1.95$$



$e[1, 3]$  is minimum for  $r = 1$ , so  $r[1, 3] = 1$

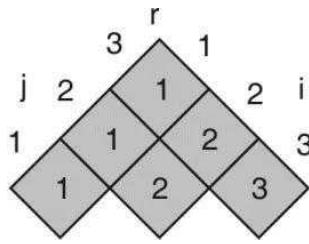
$e[2, 3]$  is minimum for  $r = 2$ , so  $r[2, 3] = 2$

$e[1, 2]$  is minimum for  $r = 1$ , so  $r[1, 2] = 1$

$e[3, 3]$  is minimum for  $r = 3$ , so  $r[3, 3] = 3$

$e[2, 2]$  is minimum for  $r = 2$ , so  $r[2, 2] = 2$

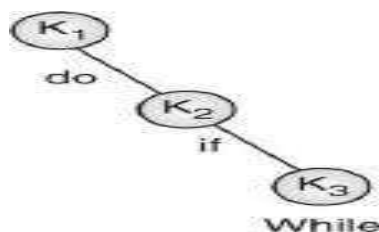




$e[1, 1]$  is minimum for  $r = 1$ , so  $r[1, 1] = 1$ . Let us now construct OBST for given data.  $r[1, 3] = 1$ , so  $k_1$  will be at the root.  $k_2, \dots, k_3$  are on right side of  $k_1$ .

$r[2, 3] = 2$ , So  $k_2$  will be the root of this sub tree.  $k_3$  will be on the right of  $k_2$ .

Thus, finally, we get.



## Greedy Technique

Activity Selection problem is an approach of selecting non-conflicting tasks based on start and end time and can be solved in  $O(N \log N)$  time using a simple greedy approach. Modifications of this problem are complex and interesting which we will explore as well. Surprisingly, if we use a Dynamic Programming approach, the time complexity will be  $O(N^3)$  that is lower performance.

The problem statement for Activity Selection is that "Given a set of  $n$  activities with their start and finish times, we need to select maximum number of non-conflicting activities that can be performed by a single person, given that the person can handle only one activity at a time." The Activity Selection problem follows Greedy approach i.e. at every step, we can make a choice that looks best at the moment to get the optimal solution of the complete problem.

Our objective is to complete maximum number of activities. So, choosing the activity which is going to finish first will leave us maximum time to adjust the later activities. This is the intuition that greedily choosing the activity with earliest finish time will give us an optimal solution. By induction on the number of choices made, making the greedy choice at every step produces an optimal solution, so we choose the activity which finishes first. If we sort elements based on their starting time, the activity with least starting time could take the maximum duration for completion, therefore we won't be able to maximise number of activities.



## Algorithm

The algorithm of Activity Selection is as follows: Activity-Selection(Activity, start, finish)

Sort Activity by finish times stored in

finishSelected = {Activity[1]}

n =

Activity.length j =

1

for i = 2 to n:

if start[i]  $\geq$  finish[j]:

Selected = Selected  $\cup$

{Activity[i]} j = i

return Selected

## Complexity

Time Complexity:

When activities are sorted by their finish time:  **$O(N)$**

When activities are not sorted by their finish time, the time complexity is  **$O(N \log N)$**  due to complexity of sorting

|       |   |   |   |   |   |    |    |
|-------|---|---|---|---|---|----|----|
| START | 1 | 3 | 2 | 0 | 5 | 8  | 11 |
| END   | 3 | 4 | 5 | 7 | 9 | 10 | 12 |

START[4] >= END[2], SELECTED

|       |   |   |   |   |   |    |    |
|-------|---|---|---|---|---|----|----|
| START | 1 | 3 | 2 | 0 | 5 | 8  | 11 |
| END   | 3 | 4 | 5 | 7 | 9 | 10 | 12 |

START[5] < END[4], REJECTED

|       |   |   |   |   |   |    |    |
|-------|---|---|---|---|---|----|----|
| START | 1 | 3 | 2 | 0 | 5 | 8  | 11 |
| END   | 3 | 4 | 5 | 7 | 9 | 10 | 12 |

START[6] >= END[4], SELECTED

0 1 2 3 4 5 6

|       |   |   |   |   |   |    |    |
|-------|---|---|---|---|---|----|----|
| START | 1 | 3 | 2 | 0 | 5 | 8  | 11 |
| END   | 3 | 4 | 5 | 7 | 9 | 10 | 12 |

SELECTED

|       |   |   |   |   |   |    |    |
|-------|---|---|---|---|---|----|----|
| START | 1 | 3 | 2 | 0 | 5 | 8  | 11 |
| END   | 3 | 4 | 5 | 7 | 9 | 10 | 12 |

START[1] >= END[0], SELECTED

|       |   |   |   |   |   |    |    |
|-------|---|---|---|---|---|----|----|
| START | 1 | 3 | 2 | 0 | 5 | 8  | 11 |
| END   | 3 | 4 | 5 | 7 | 9 | 10 | 12 |

START[2] < END[1], REJECTED

|       |   |   |   |   |   |    |    |
|-------|---|---|---|---|---|----|----|
| START | 1 | 3 | 2 | 0 | 5 | 8  | 11 |
| END   | 3 | 4 | 5 | 7 | 9 | 10 | 12 |

START[3] < END[1], REJECTED

In this example, we take the start and finish time of activities as follows:

start = [1, 3, 2, 0, 5, 8, 11]

finish = [3, 4, 5, 7, 9, 10, 12]

Sorted by their finish time, the activity 0 gets selected. As the activity 1 has starting time which is equal to the finish time of activity 0, it gets selected. Activities 2 and 3 have smaller starting time than finish time of activity 1, so they get rejected. Based on similar comparisons, activities 4 and 6 also get selected, whereas activity 5 gets rejected. In this example, in all the activities 0, 1, 4 and 6 get selected, while others get rejected.



## Optimal Merge Pattern

Merge a set of sorted files of different length into a single sorted file. We need to find an optimal solution, where the resultant file will be generated in minimum time.

If the number of sorted files are given, there are many ways to merge them into a single sorted file. This merge can be performed pair wise. Hence, this type of merging is called as 2-way merge patterns.

As, different pairings require different amounts of time, in this strategy we want to determine an optimal way of merging many files together. At each step, two shortest sequences are merged.

To merge a p-record file and a q-record file requires possibly  $p + q$  record moves, the obvious choice being, merge the two smallest files together at each step.

Two-way merge patterns can be represented by binary merge trees. Let us consider a set of  $n$  sorted files  $\{f_1, f_2, f_3, \dots, f_n\}$ . Initially, each element of this is considered as a single node binary tree. To find this optimal solution, the following algorithm is used.

### Algorithm: TREE

(n) for  $i := 1$  to  $n - 1$

    dodeclare new node

        node.leftchild := least (list)

        node.rightchild := least (list)

        node.weight) := ((node.leftchild).weight) + ((node.rightchild).weight)

        insert (list, node);

return least (list);

At the end of this algorithm, the weight of the root node represents the optimal cost.

### Example

Let us consider the given files,  $f_1, f_2, f_3, f_4$  and  $f_5$  with 20, 30, 10, 5 and 30 number of elements respectively.

If merge operations are performed according to the provided sequence, then

$M_1 = \text{merge } f_1 \text{ and } f_2 \Rightarrow 20 + 30 = 50$

$M_2 = \text{merge } M_1 \text{ and } f_3 \Rightarrow 50 + 10 = 60$

$M_3 = \text{merge } M_2 \text{ and } f_4 \Rightarrow 60 + 5 = 65$

$M_4 = \text{merge } M_3 \text{ and } f_5 \Rightarrow 65 + 30 = 95$

Hence, the total number of operations is

$50 + 60 + 65 + 95 = 270$

Now, the question arises is there any better solution?

Sorting the numbers according to their size in an ascending order, we get the following sequence –

$f_4, f_3, f_1, f_2, f_5$

Hence, merge operations can be performed on this sequence

$M_1 = \text{merge } f_4 \text{ and } f_3 \Rightarrow 5 + 10 = 15$

$M_2 = \text{merge } M_1 \text{ and } f_1 \Rightarrow 15 + 20 =$

$35$   $M_3 = \text{merge } M_2 \text{ and } f_2 \Rightarrow 35 + 30$

$= 65$   $M_4 = \text{merge } M_3 \text{ and } f_5 \Rightarrow 65 +$

$30 = 95$

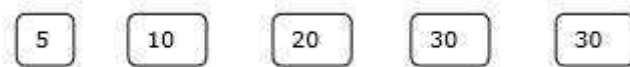
Therefore, the total number of operations is

$15 + 35 + 65 + 95 = 210$

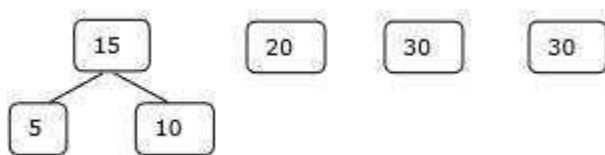
Obviously, this is better than the previous one.

In this context, we are now going to solve the problem using this algorithm.

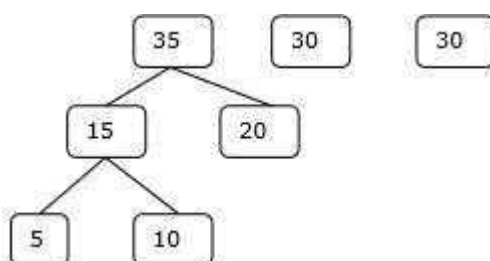
Initial Set



Step 1

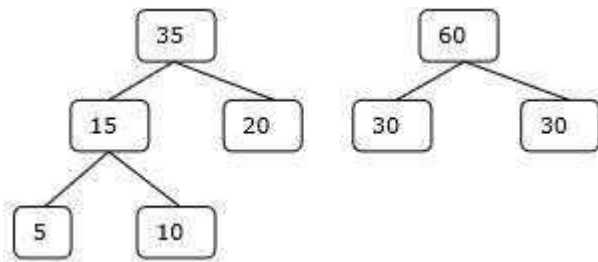


Step 2

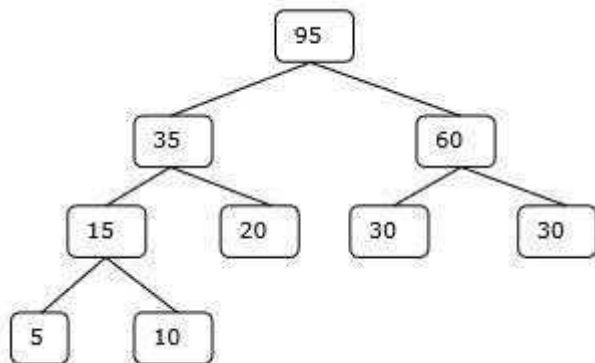




Step 3



Step 4



Hence, the solution takes  $15 + 35 + 60 + 95 = 205$  number of comparisons.

The **0/1 Knapsack Problem** is a classic optimization problem:

Given  $n$  items, each with a **weight** and a **value**, determine the **maximum total value** that can be placed in a knapsack of **capacity  $W$** , where **each item can either be included or excluded (0 or 1)**.

Formally:

- **Input:**  $n$  items, weights  $w[1..n]$ , values  $v[1..n]$ , knapsack capacity  $W$
- **Output:** Maximum possible total value without exceeding capacity.

### Purpose

- To **maximize profit** (value) while respecting a **weight limit**.
- Used in **resource allocation problems** where resources can't be divided (e.g., selecting projects, cargo loading, budgeting).
- The **dynamic programming** approach optimizes the solution by avoiding **recomputation** through subproblem reuse.

### Pseudocode (Dynamic Programming – Bottom-Up Table Method)

0/1\_Knapsack( $W$ ,  $wt[]$ ,  $val[]$ ,  $n$ ):

    Create a table  $dp[0..n][0..W]$

    For  $i = 0$  to  $n$ :

        For  $w = 0$  to  $W$ :

            if  $i == 0$  or  $w == 0$ :

$dp[i][w] = 0$

            else if  $wt[i-1] \leq w$ :

$dp[i][w] = \max(\$   
                                 $val[i-1] + dp[i-1][w - wt[i-1]],$   
                                 $dp[i-1][w]$   
                 $)$

            else:

$dp[i][w] = dp[i-1][w]$

    return  $dp[n][W]$

### Explanation:

- $dp[i][w]$  = maximum value for first  $i$  items with capacity  $w$ .
- If current item's weight  $\leq$  remaining capacity, we choose max of:
  1. **Including** the item  $\rightarrow$  value =  $val[i-1] + dp[i-1][w - wt[i-1]]$
  2. **Excluding** the item  $\rightarrow$  value =  $dp[i-1][w]$
- Else: we **skip** the item.



### Example Problem

Item 1:  $w_1 = 1, v_1 = 6$

Item 2:  $w_2 = 2, v_2 = 10$

Item 3:  $w_3 = 3, v_3 = 12$

Capacity  $W = 5$ .

We build  $dp[i][c]$  for  $i = 0..3, c = 0..5$ .

Initial table (all zeros)

| i \ c | 0 | 1 | 2 | 3 | 4 | 5 |
|-------|---|---|---|---|---|---|
| 0     | 0 | 0 | 0 | 0 | 0 | 0 |
| 1     | 0 | 0 | 0 | 0 | 0 | 0 |
| 2     | 0 | 0 | 0 | 0 | 0 | 0 |
| 3     | 0 | 0 | 0 | 0 | 0 | 0 |

Fill row  $i = 1$  (Item 1:  $w=1, v=6$ )

We compute  $dp[1][c]$  for  $c = 0..5$ .

$c = 0$ :  $dp[1][0] = dp[0][0] = 0$ .

$c = 1$ :  $w_1 (=1) \leq 1 \rightarrow$  compare:

not take  $\rightarrow dp[0][1] = 0$

take  $\rightarrow dp[0][1-1] + v_1 = dp[0][0] + 6 = 0 + 6 = 6$   
 $\rightarrow dp[1][1] = \max(0, 6) = 6$ .

$c = 2$ :  $w_1 \leq 2 \rightarrow$  compare:

$dp[0][2] = 0$

$dp[0][2-1] + 6 = dp[0][1] + 6 = 0 + 6 = 6$   
 $\rightarrow dp[1][2] = 6$ .

$c = 3$ : similarly  $dp[1][3] = 6$  (take item 1 fits).

$c = 4$ :  $dp[1][4] = 6$ .

$c = 5$ :  $dp[1][5] = 6$ .

Row after  $i=1$ :

| i \ c | 0 | 1 | 2 | 3 | 4 | 5 |
|-------|---|---|---|---|---|---|
| 0     | 0 | 0 | 0 | 0 | 0 | 0 |

| i \ c | 0 | 1 | 2 | 3 | 4 | 5 |
|-------|---|---|---|---|---|---|
| 1     | 0 | 6 | 6 | 6 | 6 | 6 |
| 2     | 0 | 0 | 0 | 0 | 0 | 0 |
| 3     | 0 | 0 | 0 | 0 | 0 | 0 |

Fill row  $i = 2$  (Item 2:  $w=2$ ,  $v=10$ )

Compute  $dp[2][c]$  for  $c = 0..5$ .

$c = 0$ :  $dp[2][0] = dp[1][0] = 0$ .

$c = 1$ :  $w_2(=2) > 1 \rightarrow$  cannot take, so  $dp[2][1] = dp[1][1] = 6$ .

$c = 2$ :  $w_2 \leq 2 \rightarrow$  compare:

not take  $\rightarrow dp[1][2] = 6$

take  $\rightarrow dp[1][2-2] + 10 = dp[1][0] + 10 = 0 + 10 = 10$   
 $\rightarrow dp[2][2] = \max(6, 10) = 10$ .

$c = 3$ :  $w_2 \leq 3 \rightarrow$  compare:

not take  $\rightarrow dp[1][3] = 6$

take  $\rightarrow dp[1][3-2] + 10 = dp[1][1] + 10 = 6 + 10 = 16$   
 $\rightarrow dp[2][3] = \max(6, 16) = 16$ .

$c = 4$ : compare:

not take  $\rightarrow dp[1][4] = 6$

take  $\rightarrow dp[1][4-2] + 10 = dp[1][2] + 10 = 6 + 10 = 16$   
 $\rightarrow dp[2][4] = 16$ .

$c = 5$ : compare:

not take  $\rightarrow dp[1][5] = 6$

take  $\rightarrow dp[1][5-2] + 10 = dp[1][3] + 10 = 6 + 10 = 16$   
 $\rightarrow dp[2][5] = 16$ .

Row after  $i=2$ :

| i \ c | 0 | 1 | 2 | 3 | 4 | 5 |
|-------|---|---|---|---|---|---|
| 0     | 0 | 0 | 0 | 0 | 0 | 0 |



| i \ c | 0 | 1 | 2  | 3  | 4  | 5  |
|-------|---|---|----|----|----|----|
| 1     | 0 | 6 | 6  | 6  | 6  | 6  |
| 2     | 0 | 6 | 10 | 16 | 16 | 16 |
| 3     | 0 | 0 | 0  | 0  | 0  | 0  |

Fill row  $i = 3$  (Item 3:  $w=3, v=12$ )

Compute  $dp[3][c]$  for  $c = 0..5$ .

$c = 0$ :  $dp[3][0] = dp[2][0] = 0$ .

$c = 1$ :  $w_3(=3) > 1 \rightarrow dp[3][1] = dp[2][1] = 6$ .

$c = 2$ :  $w_3 > 2 \rightarrow dp[3][2] = dp[2][2] = 10$ .

$c = 3$ :  $w_3 \leq 3 \rightarrow$  compare:

not take  $\rightarrow dp[2][3] = 16$

take  $\rightarrow dp[2][3-3] + 12 = dp[2][0] + 12 = 0 + 12 = 12$

$\rightarrow dp[3][3] = \max(16, 12) = 16$  (better to keep items  $\{1,2\}$  rather than just item3)

$c = 4$ : compare:

not take  $\rightarrow dp[2][4] = 16$

take  $\rightarrow dp[2][4-3] + 12 = dp[2][1] + 12 = 6 + 12 = 18$

$\rightarrow dp[3][4] = \max(16, 18) = 18$ .

$c = 5$ : compare:

not take  $\rightarrow dp[2][5] = 16$

take  $\rightarrow dp[2][5-3] + 12 = dp[2][2] + 12 = 10 + 12 = 22$

$\rightarrow dp[3][5] = \max(16, 22) = 22$ .

Final table:

| i \ c | 0 | 1 | 2 | 3 | 4 | 5 |
|-------|---|---|---|---|---|---|
| 0     | 0 | 0 | 0 | 0 | 0 | 0 |

| i\c | 0 | 1 | 2  | 3  | 4  | 5  |
|-----|---|---|----|----|----|----|
| 1   | 0 | 6 | 6  | 6  | 6  | 6  |
| 2   | 0 | 6 | 10 | 16 | 16 | 16 |
| 3   | 0 | 6 | 10 | 16 | 18 | 22 |

So the answer (maximum total value for capacity 5) is  $dp[3][5] = 22$ .

### Backtracking (which items were chosen) — step by step

Start at  $(i, c) = (3, 5)$  and compare  $dp[i][c]$  with  $dp[i-1][c]$ :

$(3,5)$ :  $dp[3][5] = 22$ ,  $dp[2][5] = 16$ . They differ → **item 3 was taken**.

Record Item 3 ( $w_3 = 3$ ,  $v_3 = 12$ ).

Reduce capacity:  $c = 5 - w_3 = 5 - 3 = 2$ . Move to  $i = 2$ .

$(2,2)$ :  $dp[2][2] = 10$ ,  $dp[1][2] = 6$ . They differ → **item 2 was taken**.

Record Item 2 ( $w_2 = 2$ ,  $v_2 = 10$ ).

Reduce capacity:  $c = 2 - w_2 = 0$ . Move to  $i = 1$ .

$c = 0$ , stop (no remaining capacity).

Chosen items: **Item 2 and Item 3**.

Check: total weight =  $2 + 3 = 5 (\leq W)$ , total value =  $10 + 12 = 22$  (matches  $dp[3][5]$ ).

### Time & Space Complexity

- **Time Complexity:**  
We fill  $n \times W$  table →  **$O(nW)$** 
  - $n$  = number of items
  - $W$  = knapsack capacity
- **Space Complexity:**
  - Table method →  **$O(nW)$**
  - Optimized 1D DP (rolling array) →  **$O(W)$**



## UNIT IV STATE SPACE SEARCH ALGORITHMS

Backtracking: n-Queens problem - Hamiltonian Circuit Problem - Subset Sum Problem – Graph colouring problem Branch and Bound: Solving 15-Puzzle problem - Assignment problem - Knapsack Problem - Travelling Salesman Problem

### Backtracking

#### N queen Problem

N - Queens problem is to place n - queens in such a manner on an n x n chessboard that no queens attack each other by being in the same row, column or diagonal.

It can be seen that for  $n = 1$ , the problem has a trivial solution, and no solution exists for  $n = 2$  and  $n = 3$ . So first we will consider the 4 queens problem and then generate it to n - queens problem.

Given a 4 x 4 chessboard and number the rows and column of the chessboard 1 through 4.

|   | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 |   |   |   |   |
| 2 |   |   |   |   |
| 3 |   |   |   |   |
| 4 |   |   |   |   |

4x4 chessboard

Since, we have to place 4 queens such as  $q_1$   $q_2$   $q_3$  and  $q_4$  on the chessboard, such that no two queens attack each other. In such a conditional each queen must be placed on a different row, i.e., we put queen " $i$ " on row " $i$ ."

Now, we place queen  $q_1$  in the very first acceptable position (1, 1). Next, we put queen  $q_2$  so that both these queens do not attack each other. We find that if we place  $q_2$  in column 1 and 2, then the dead end is encountered. Thus the first acceptable position for  $q_2$  in column 3, i.e. (2, 3) but then no position is left for placing queen ' $q_3$ ' safely. So we backtrack one step and place the queen ' $q_2$ ' in (2, 4), the next best possible solution. Then we obtain the position for placing ' $q_3$ ' which is (3, 2). But later this position also leads to a dead end, and no place is found where ' $q_4$ ' can be placed safely. Then we have to backtrack till ' $q_1$ ' and place it to (1, 2) and then all other queens are placed safely by moving  $q_2$  to (2, 4),  $q_3$  to (3, 1) and  $q_4$  to (4, 3). That is, we get the solution (2, 4, 1, 3). This is one possible solution for the 4-queens problem. For another possible solution, the whole method is repeated for all partial solutions. The other solutions for 4 - queens problems is (3, 1, 4, 2) i.e.

|   | 1     | 2     | 3     | 4     |
|---|-------|-------|-------|-------|
| 1 |       |       | $q_1$ |       |
| 2 | $q_2$ |       |       |       |
| 3 |       |       |       | $q_3$ |
| 4 |       | $q_4$ |       |       |

The implicit tree for 4 - queen problem for a solution (2, 4, 1, 3) is as follows:

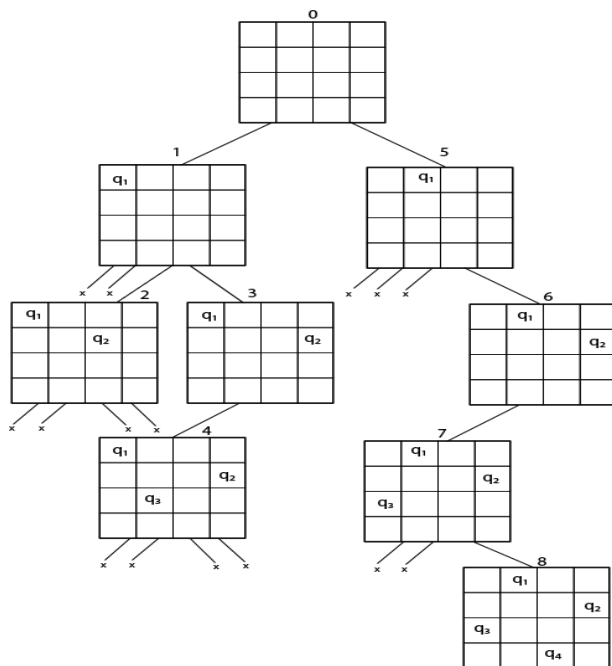
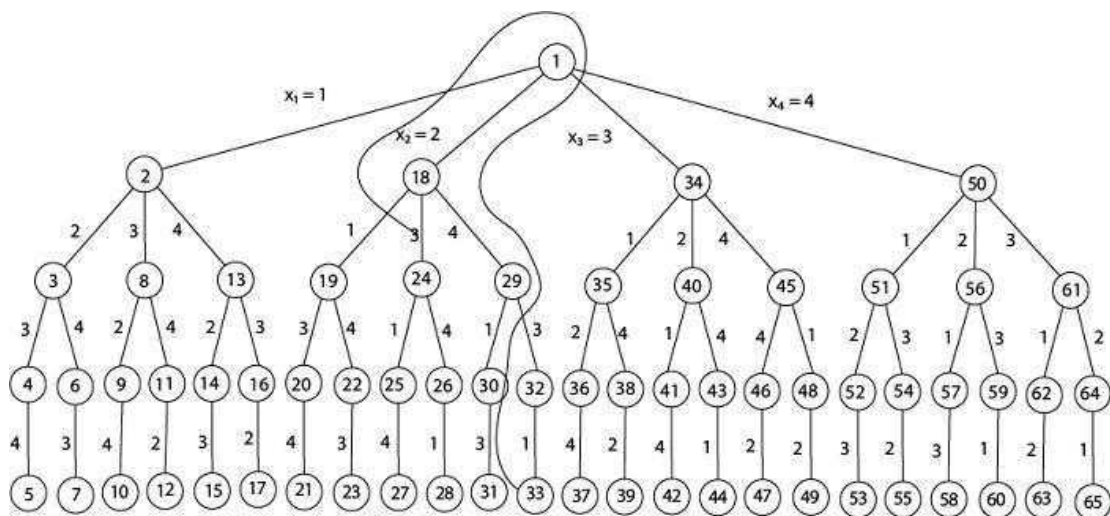


Fig shows the complete state space for 4 - queens problem. But we can use backtracking method to generate the necessary node and stop if the next node violates the rule, i.e., if two queens are attacking.



#### 4 - Queens solution space with nodes numbered in DFS

It can be seen that all the solutions to the 4 queens problem can be represented as 4 - tuples  $(x_1, x_2, x_3, x_4)$  where  $x_i$  represents the column on which queen " $q_i$ " is placed.

One possible solution for 8 queens problem is shown in fig:



|   | 1              | 2              | 3              | 4              | 5              | 6              | 7              | 8 |
|---|----------------|----------------|----------------|----------------|----------------|----------------|----------------|---|
| 1 |                |                |                | q <sub>1</sub> |                |                |                |   |
| 2 |                |                |                |                |                | q <sub>2</sub> |                |   |
| 3 |                |                |                |                |                |                | q <sub>3</sub> |   |
| 4 |                | q <sub>4</sub> |                |                |                |                |                |   |
| 5 |                |                |                |                |                | q <sub>5</sub> |                |   |
| 6 | q <sub>6</sub> |                |                |                |                |                |                |   |
| 7 |                |                | q <sub>7</sub> |                |                |                |                |   |
| 8 |                |                |                |                | q <sub>8</sub> |                |                |   |

Thus, the solution **for** 8 -queen problem **for** (4, 6, 8, 2, 7, 1, 3, 5).

If two queens are placed at position (i, j) and (k, l).

Then they are on same diagonal only **if**  $(i - j) = k - l$  or  $i + j = k + l$ .

The first equation implies that  $j - l = i - k$ .

The second equation implies that  $j - l = k - i$ .

Therefore, two queens lie on the duplicate diagonal **if** and only **if**  $|j-l|=|i-k|$

Place (k, i) returns a Boolean value that is true if the kth queen can be placed in column i. It tests both whether i is distinct from all previous costs  $x_1, x_2, \dots, x_{k-1}$  and whether there is no other queen on the same diagonal.

Using place, we give a precise solution to then n- queens problem.

Place (k, i)

{

For j  $\leftarrow$  1 to k - 1

**do if**  $(x[j] = i)$

or  $(\text{Abs } x[j] - i) = (\text{Abs } (j - k))$

then **return false**;

**return true**;

}

Place (k, i) return true if a queen can be placed in the kth row and ith column otherwise return is false.  $x[]$  is a global array whose final k - 1 values have been set. Abs (r) returns the absolute value of r.

N - Queens (k, n)2.

{

For i  $\leftarrow$  1 to n

**do if** Place (k, i) then5. {

$x[k] \leftarrow i$ ;

**if**  $(k == n)$  then

write  $(x[1] \dots \dots \dots n)$ );

**else**

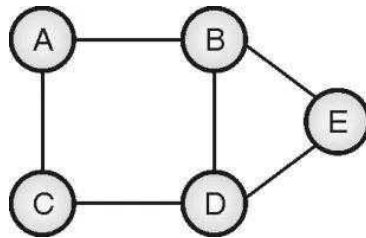
N - Queens (k + 1, n); } }

## Hamiltonian Circuit

The **Hamiltonian cycle** is the cycle in the graph which visits all the vertices in graph exactly once and terminates at the starting node. It may not include all the edges

- **The** Hamiltonian cycle problem is the problem of finding a Hamiltonian cycle in a graph if there exists any such cycle.

- The input to the problem is an undirected, connected graph. For the graph shown in Figure (a), a path A – B – E – D – C – A forms a Hamiltonian cycle. It visits all the vertices exactly once, but does not visit the edges <B, D>.



The Hamiltonian cycle problem is also both, decision problem and an optimization problem. A decision problem is stated as, “Given a path, is it a Hamiltonian cycle of the graph?”.

1st and (n – 1)th vertex must be adjacent (nth of cycle is the initial vertex itself).

Vertex i must not appear in the first (i – 1) vertices of any path.

With the adjacency matrix representation of the graph, the adjacency of two vertices can be verified in constant time.

## Algorithm

HAMILTONIAN (i)

// Description : Solve Hamiltonian cycle problem using backtracking.

// Input : Undirected, connected graph  $G = \langle V, E \rangle$  and initial vertex i

// Output : Hamiltonian cycle

**if**

FEASIBLE(i)

**then**

**if**

(i == n - 1)

**then**

Print V[0... n – 1]

**else**

j ← 2

**while**(j

≤ n) **do**



```

V[i] ← j HAMILTONIAN(i + 1)
j ← j + 1 end
end

```

```

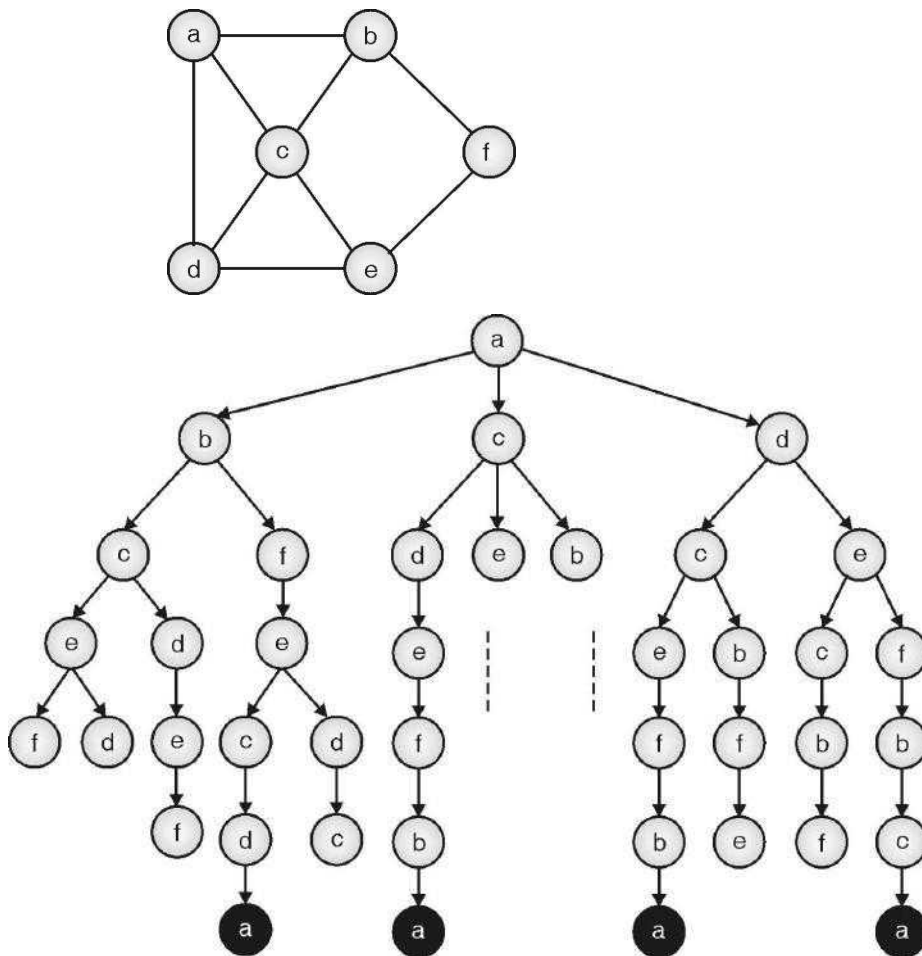
end
function
FEASIBLE(i)
flag ← 1
for
j ← 1 to i - 1
doif
Adjacent(Vi, Vj)
then
flag ← 0
endend if
Adjacent (Vi, Vi-1)
then
flag ← 1
else
flag ← 0
end return flag

```

### Complexity Analysis

Looking at the state space graph, in worst case, total number of nodes in tree would be,  $T(n) = 1 + (n - 1) + (n - 1)^2 + (n - 1)^3 + \dots + (n - 1)^{n-1}$   
 $= \frac{(n-1)n-1n-2}{n-1}$   
 $T(n) = O(n^n)$ . Thus, the Hamiltonian cycle algorithm runs in exponential time.

**Example: Find the Hamiltonian cycle by using the backtracking approach for a given graph.**



The backtracking approach uses a state-space tree to check if there exists a Hamiltonian cycle in the graph. Figure (g) shows the simulation of the Hamiltonian cycle algorithm. For simplicity, we have not explored all possible paths, the concept is self-explanatory. It is not possible to include all the paths in the graph, so few of the successful and unsuccessful paths are traced in the graph. Black nodes indicate the Hamiltonian cycle.



## Subset Sum Problem

**Sum of Subsets Problem:** Given a set of positive integers, find the combination of numbers that sum to given value M.

Sum of subsets problem is analogous to the **knapsack problem**. The Knapsack Problem tries to fill the knapsack using a given set of items to maximize the profit. Items are selected in such a way that the total weight in the knapsack does not exceed the capacity of the knapsack. The inequality condition in the knapsack problem is replaced by equality in the sum of subsets problem.

Given the set of n positive integers,  $W = \{w_1, w_2, \dots, w_n\}$ , and given a positive integer M, the sum of the subset problem can be formulated as follows (where  $w_i$  and M correspond to item weights and knapsack capacity in the knapsack problem):

$$\sum_{i=1}^n w_i x_i = M$$

Where,

$$x_i \in \{0, 1\}$$

Numbers are sorted in ascending order, such that  $w_1 < w_2 < w_3 < \dots < w_n$ . The solution is often represented using the solution vector X. If the  $i$ th item is included, set  $x_i$  to 1 else set it to 0. In each iteration, one item is tested. If the inclusion of an item does not violate the constraint of the problem, add it. Otherwise, backtrack, remove the previously added item, and continue the same procedure for all remaining items. The solution is easily described by the state space tree. Each left edge denotes the inclusion of  $w_i$  and the right edge denotes the exclusion of  $w_i$ . Any path from the root to the leaf forms a subset. A state-space tree for  $n = 3$  is demonstrated in Fig. (a).

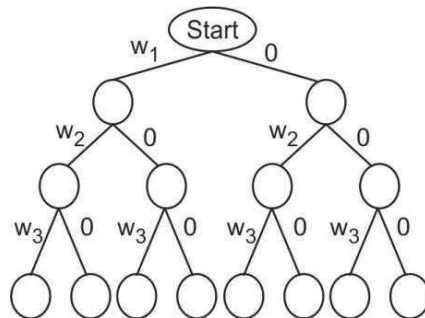


Fig. (a): State space tree for  $n = 3$

### Algorithm for Sum of subsets

The algorithm for solving the sum of subsets problem using recursion is stated below:

```

Algorithm sumofsubsets(s,k,r)
{
  X[k]:=1;
  if (s+w[k]=m) then write (x[1:k]); // subset found
  else
    if (s+w[k]+w[k+1]<=m) then
      sumofsubsets(s+w[k],k+1,r-w[k]);

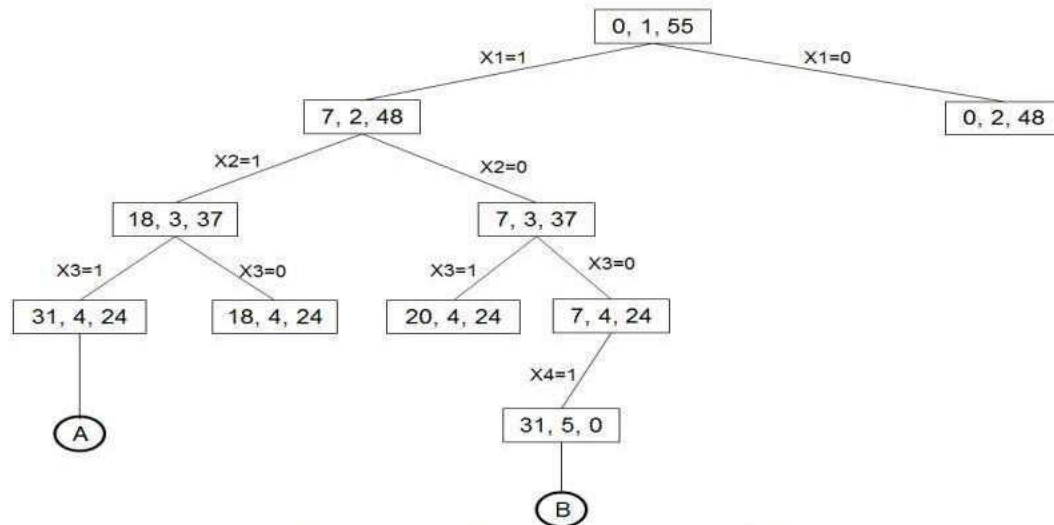
  //generate right child and evaluate Bk.

  if ((s+r-w[k]>=m) and (s+w[k+1]<=m)) then
  {
    X[k]:=0;
    sumofsubsets(s,k+1,r-w[k]);
  }
}

```

## Examples

Ex)  $n=4$ ,  $(w_1, w_2, w_3, w_4) = (7, 11, 13, 24)$  and  $m=31$   
 Solution Vector =  $(x[1], x[2], x[3], x[4])$



Portion of state space Tree

Solution A =  $\{1, 1, 1, 0\}$   
 Solution B =  $\{1, 0, 0, 1\}$



## Graph Colouring

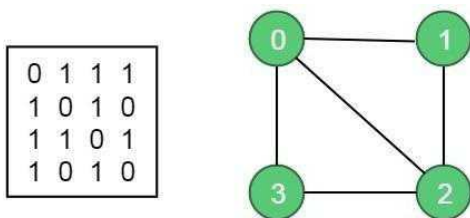
In this problem, an undirected graph is given. There is also provided  $m$  colors. The problem is to find if it is possible to assign nodes with  $m$  different colors, such that no two adjacent vertices of the graph are of the same colors. If the solution exists, then display which color is assigned on which vertex.

Starting from vertex 0, we will try to assign colors one by one to different nodes. But before assigning, we have to check whether the color is safe or not. A color is not safe whether adjacent vertices are containing the same color.

Input and

Output

The adjacency matrix of a graph  $G(V, E)$  and an integer  $m$ , which indicates the maximum number of colors that can be used.



Let the maximum color  $m =$

3. Output:

This algorithm will return which node will be assigned with which color. If the solution is not possible, it will return false.

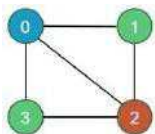
For this input the assigned colors are:

Node 0 -> color 1

Node 1 -> color 2

Node 2 -> color 3

Node 3 -> color 2



Algorithm

**isValid(vertex, colorList, col)**

**Input** – Vertex, colorList to check, and color, which is trying to assign.

**Output** – True if the color assigning is valid, otherwise false.

Begin

for all vertices  $v$  of the graph, do

if there is an edge between  $v$  and  $i$ , and  $col = colorList[i]$ ,

then return false

done return

true

End

### graphColoring(colors, colorList, vertex)

**Input** – Most possible colors, the list for which vertices are colored with which color, and the starting vertex.

**Output** – True, when colors are assigned, otherwise false.

Begin

if all vertices are checked, then

return true

for all colors col from available colors,

do if isValid(vertex, color, col), then

add col to the colorList for vertex

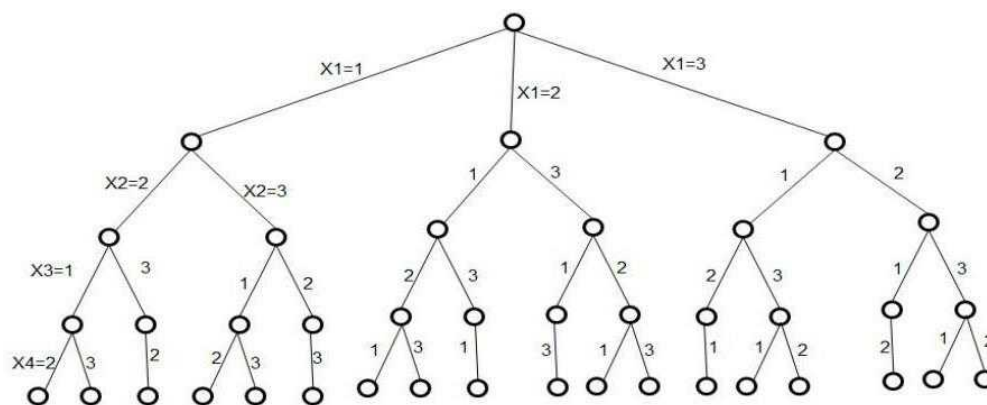
if graphColoring(colors, colorList, vertex+1) = true, then

return true

remove color for vertex

done

return false



A 4-node graph and all possible 3-colorings

End

### Branch and Bound

#### Solving 15 puzzle Problem (LCBB)

The problem consists of 15 numbered (0-15) tiles on a square box with 16 tiles (one tile is blank or empty). The objective of this problem is to change the arrangement of initial node to goal node by using series of legal moves.

The Initial and Goal node arrangement is shown by following figure.

|   |   |    |    |
|---|---|----|----|
| 1 | 2 | 4  | 15 |
| 2 |   | 5  | 12 |
| 7 | 6 | 11 | 14 |
| 8 | 9 | 10 | 13 |

Initial Arrangement

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 |    |

Final Arrangement

In initial node four moves are possible. User can move any one of the tile like 2, or 3, or 5, or 6 to the empty tile. From this we have four possibilities to move from initial node.

The legal moves are for adjacent tile number is left, right, up, down, ones at a time.

Each and every move creates a new arrangement, and this arrangement is called state of puzzle problem. By using different states, a state space tree diagram is created, in which edges are labeled according to the direction in which the empty space moves.

The state space tree is very large because it can be 16! Different arrangements.

In state space tree, nodes are numbered as per the level. In each level we must calculate the value or cost of each node by using given formula:

$$C(x) = f(x) + g(x),$$

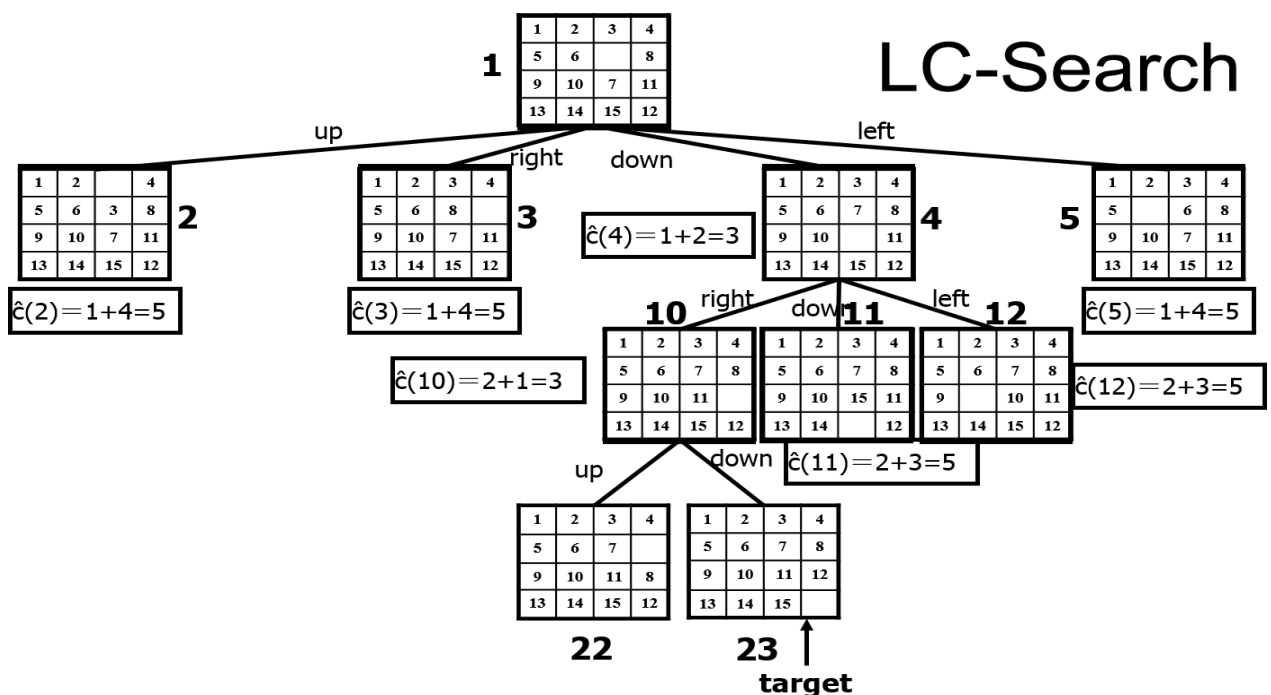
$f(x)$  is length of path from root or initial node to node  $x$ ,

$g(x)$  is estimated length of path from  $x$  downward to the goal node. Number of non blank tile not in their correct position.

$$C(x) < \text{Infinity. (initially set bound).}$$

Each time node with smallest cost is selected for further expansion towards goal node. This node become the e-node.

State Space tree with node cost is shown in diagram.





## Assignment Problem

### Problem Statement

Let's first define a job assignment problem. In a standard version of a job assignment problem, there can be jobs and workers. To keep it simple, we're taking jobs and workers in our example:

|   | Job 1 | Job 2 | Job 3 |
|---|-------|-------|-------|
| A | 9     | 3     | 4     |
| B | 7     | 8     | 4     |
| C | 10    | 5     | 2     |

We can assign any of the available jobs to any worker with the condition that if a job is assigned to a worker, the other workers can't take that particular job. We should also notice that each job has some cost associated with it, and it differs from one worker to another.

Here the main aim is to complete all the jobs by assigning one job to each worker in such a way that the sum of the cost of all the jobs should be minimized.

### Branch and Bound Algorithm Pseudocode

Now let's discuss how to solve the job assignment problem using a branch and bound algorithm. Let's see the pseudocode first:

---

**Algorithm 1:** Job Assignment Problem Using Branch And Bound

---

**Data:** Input cost matrix  $M[][]$

**Result:** Assignment of jobs to each worker according to optimal cost

**Function**  $MinCost(M[][])$

**while** *True* **do**

$E = LeastCost();$

**if** *E is a leaf node* **then**

$print();$

**return;**

**end**

**for** *each child S of E* **do**

$Add(S);$

$S \rightarrow parent = E;$

**end**

**end**

---

Here, is the input cost matrix that contains information like the number of available jobs, a list of available workers, and the associated cost for each job. The function  $MinCost()$  maintains a list of active nodes. The function  $Leastcost()$  calculates the minimum cost of the active node at each level of the tree. After finding the node with minimum cost, we remove the node from the list of active nodes and return it.

We're using the  $add()$  function in the pseudocode, which calculates the cost of a particular node and adds it to the list of active nodes.

In the search space tree, each node contains some information, such as cost, a total number of jobs, as well as a total number of workers.

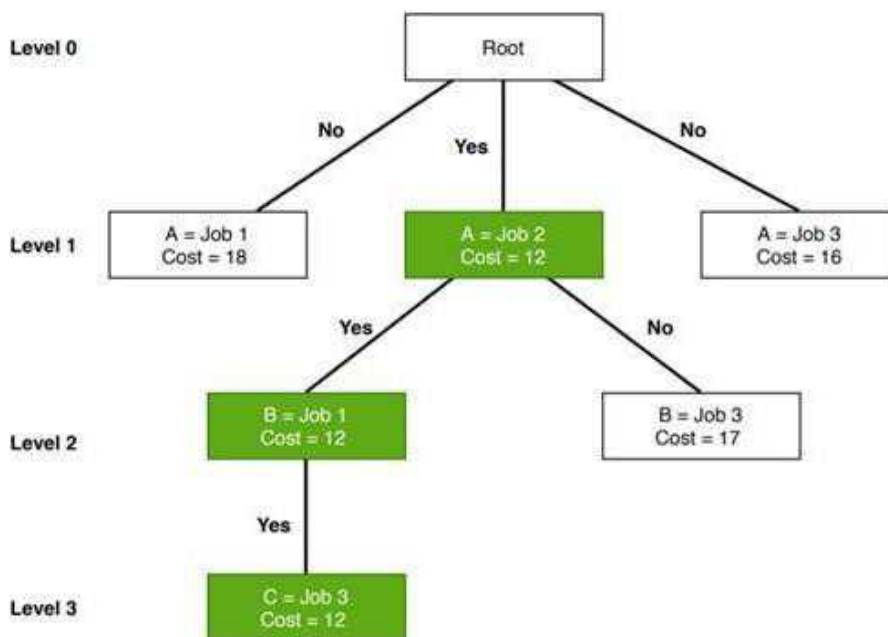
Now let's run the algorithm on the sample example we've created:

Initially, we've 3 jobs available. The worker *A* has the option to take any of the available jobs. So at level 1, we assigned all the available jobs to the worker *A* and calculated the cost. We can see that when we assigned jobs 2 to the worker *A*, it gives the lowest cost in level 1 of the search space tree. **So we assign the job 2 to worker *A* and continue the algorithm. "Yes" indicates that this is currently optimal cost.**

After assigning the job 2 to worker *A*, we still have two open jobs. Let's consider worker *B* now. We're trying to assign either job 1 or 3 to worker *B* to obtain optimal cost.

Either we can assign the job 1 or 3 to worker *B*. **Again we check the cost and assign job 1 to worker *B* as it is the lowest in level 2.**

**Finally, we assign the job 3 to worker *C*, and the optimal cost is 12.**



### Advantages

In a branch and bound algorithm, we don't explore all the nodes in the tree. That's why **the time complexity of the branch and bound algorithm is less when compared with other algorithms.**

If the problem is not large and if we can do the branching in a reasonable amount of time, **it finds an optimal solution for a given problem.**

The branch and bound algorithm find a minimal path to reach the optimal solution for a given problem. **It doesn't repeat nodes while exploring the tree.**

### Disadvantages

**The branch and bound algorithm are time-consuming.** Depending on the size of the given problem, the number of nodes in the tree can be too large in the worst case.

## Knapsack Problem using branch and bound

### Problem Statement

We are given a set of  $n$  objects which have each have a value  $v_i$  and a weight  $w_i$ . The objective of the 0/1 Knapsack problem is to find a subset of objects such that the total value is

maximized, and

the sum of weights of the objects does not exceed a given threshold  $W$ . An important condition here is that one can either take the entire object or leave it. It is not possible to take a fraction of the object.

Consider an example where  $n = 4$ , and the values are given by  $\{10, 12, 12, 18\}$  and the weights given by  $\{2, 4, 6, 9\}$ . The maximum weight is given by  $W = 15$ . Here, the solution to the problem will be including the first, third and the fourth objects.

Here, the procedure to solve the problem is as follows are:

- Calculate the cost function and the Upper bound for the two children of each node. Here, the  $(i + 1)^{\text{th}}$  level indicates whether the  $i^{\text{th}}$  object is to be included or not.
- If the cost function for a given node is greater than the upper bound, then the node need not be explored further. Hence, we can kill this node. Otherwise, calculate the upper bound for this node. If this value is less than  $U$ , then replace the value of  $U$  with this value. Then, kill all unexplored nodes which have cost function greater than this value.
- The next node to be checked after reaching all nodes in a particular level will be the one with the least cost function value among the unexplored nodes.
- While including an object, one needs to check whether the adding the object crossed the threshold. If it does, one has reached the terminal point in that branch, and all the succeeding objects will not be included.

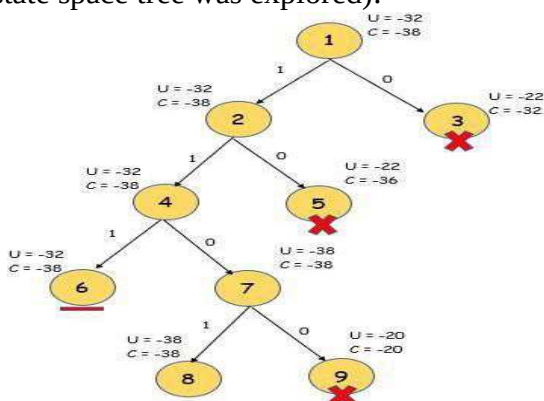
### Time and Space Complexity

Even though this method is more efficient than the other solutions to this problem, its worst case time complexity is still given by  $O(2^n)$ , in cases where the entire tree has to be explored. However, in its best case, only one path through the tree will have to be explored, and hence its best case time complexity is given by  $O(n)$ . Since this method requires the creation of the state space tree, its space complexity will also be exponential.

### Solving an Example

Consider the problem with  $n = 4$ ,  $V = \{10, 10, 12, 18\}$ ,  $w = \{2, 4, 6, 9\}$  and  $W = 15$ . Here, we calculate the initial upper bound to be  $U = 10 + 10 + 12 = 32$ . Note that the 4th object cannot be included here, since that would exceed  $W$ . For the cost, we add  $3/9^{\text{th}}$  of the final value, and hence the cost function is 38. Remember to negate the values after calculation before comparison.

After calculating the cost at each node, kill nodes that do not need exploring. Hence, the final state space tree will be as follows (Here, the number of the node denotes the order in which the state space tree was explored):





Note here that node 3 and node 5 have been killed after updating U at node 7. Also, node 6 is not explored further, since adding any more weight exceeds the threshold. At the end, only nodes 6 and 8 remain. Since the value of U is less for node 8, we select this node. Hence the solution is {1, 1, 0, 1}, and we can see here that the total weight is exactly equal to the threshold value in this case.

### Travelling salesman problem

- Travelling Salesman Problem (TSP) is an interesting problem. Problem is defined as “given n cities and distance between each pair of cities, find out the path which visits each city exactly once and come back to starting city, with the constraint of minimizing the travelling distance.”

- TSP has many practical applications. It is used in network design, and transportation route design. The objective is to minimize the distance. We can start tour from any random city and visit other cities in any order. With n cities, n! different permutations are possible. Exploring all paths using brute force attacks may not be useful in real life applications.

### LCBB using Static State Space Tree for Travelling Salseman Problem

- Branch and bound is an effective way to find better, if not best, solution in quick time by pruning some of the unnecessary branches of search tree.

- It works as follow:

Consider directed weighted graph  $G = (V, E, W)$ , where node represents cities and weighted directed edges represents direction and distance between two cities.

- Initially, graph is represented by cost matrix C, where

$C_{ij}$  = cost of edge, if there is a direct path from city i to city j  
 $C_{ij} = \infty$ , if there is no direct path from city i to city j.

- Convert cost matrix to reduced matrix by subtracting minimum values from appropriate rows and columns, such that each row and column contains at least one zero entry.

- Find cost of reduced matrix. Cost is given by summation of subtracted amount from the cost matrix to convert it in to reduce matrix.

- Prepare state space tree for the reduce matrix

- Find least cost valued node A (i.e. E-node), by computing reduced cost node matrix with every remaining node.

- If  $\langle i, j \rangle$  edge is to be included, then do following :

- Set all values in row i and all values in column j of A to  $\infty$

- Set  $A[j, 1] = \infty$

- Reduce A again, except rows and columns having all  $\infty$  entries.

- Compute the cost of newly created reduced matrix as,  $\text{Cost} = L + \text{Cost}(i, j) + r$

Where, L is cost of original reduced cost matrix and r is  $A[i, j]$ .

- If all nodes are not visited then go to step 4. Reduction procedure is described below

#### Raw Reduction:

Matrix M is called reduced matrix if each of its row and column has at least one zero entry or entire row or entire column has  $\infty$  value. Let M represents the distance matrix of 5 cities. M can be reduced as follow:

$M_{\text{RowRed}} = \{M_{ij} - \min \{M_{ij} \mid 1 \leq j \leq n, \text{ and } M_{ij} < \infty\}\}$  Consider the following distance matrix:

$$M = \begin{array}{|c|c|c|c|c|} \hline \infty & 20 & 30 & 10 & 11 \\ \hline 15 & \infty & 16 & 4 & 2 \\ \hline 3 & 5 & \infty & 2 & 4 \\ \hline 19 & 6 & 18 & \infty & 3 \\ \hline 16 & 4 & 7 & 16 & \infty \\ \hline \end{array}$$

Find the minimum element from each row and subtract it from each cell of matrix.

$$M = \begin{array}{|c|c|c|c|c|} \hline \infty & 20 & 30 & 10 & 11 \\ \hline 15 & \infty & 16 & 4 & 2 \\ \hline 3 & 5 & \infty & 2 & 4 \\ \hline 19 & 6 & 18 & \infty & 3 \\ \hline 16 & 4 & 7 & 16 & \infty \\ \hline \end{array} \begin{array}{l} \rightarrow 10 \\ \rightarrow 2 \\ \rightarrow 2 \\ \rightarrow 3 \\ \rightarrow 4 \end{array}$$

Reduced matrix would be:

$$M_{\text{RowRed}} = \begin{array}{|c|c|c|c|c|} \hline \infty & 10 & 20 & 0 & 1 \\ \hline 13 & \infty & 14 & 2 & 0 \\ \hline 1 & 3 & \infty & 0 & 2 \\ \hline 16 & 3 & 15 & \infty & 0 \\ \hline 12 & 0 & 3 & 12 & \infty \\ \hline \end{array}$$

Row reduction cost is the summation of all the values subtracted from each rows: Row reduction cost (M) = 10 + 2 + 2 + 3 + 4 = 21

### Column reduction:

Matrix  $M_{\text{RowRed}}$  is row reduced but not the column reduced. Matrix is called column reduced if each of its column has at least one zero entry or all  $\infty$  entries.

$$M_{\text{ColRed}} = \{M_{ji} - \min \{M_{ji} \mid 1 \leq j \leq n, \text{ and } M_{ji} < \infty\}\}$$

To reduced above matrix, we will find the minimum element from each column and subtract it from each cell of matrix.

$$M_{\text{RowRed}} = \begin{array}{|c|c|c|c|c|} \hline \infty & 10 & 20 & 0 & 1 \\ \hline 13 & \infty & 14 & 2 & 0 \\ \hline 1 & 3 & \infty & 0 & 2 \\ \hline 16 & 3 & 15 & \infty & 0 \\ \hline 12 & 0 & 3 & 12 & \infty \\ \hline \end{array}$$

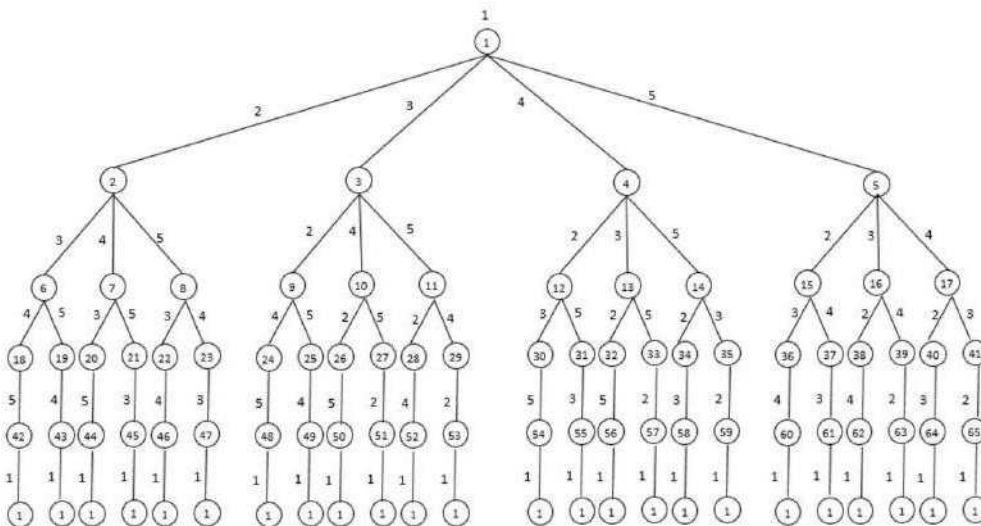
$$\begin{array}{c} \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ \hline 1 \quad 0 \quad 3 \quad 0 \quad 0 \end{array}$$

Column reduced matrix  $M_{\text{ColRed}}$  would be:

$$M_{\text{ColRed}} = \begin{array}{|c|c|c|c|c|} \hline \infty & 10 & 17 & 0 & 1 \\ \hline 12 & \infty & 11 & 2 & 0 \\ \hline 0 & 3 & \infty & 0 & 2 \\ \hline 15 & 3 & 12 & \infty & 0 \\ \hline 11 & 0 & 0 & 12 & \infty \\ \hline \end{array}$$

Each row and column of  $M_{\text{ColRed}}$  has at least one zero entry, so this matrix is reduced matrix. Column reduction cost (M) = 1 + 0 + 3 + 0 + 0 = 4

State space tree for 5 city problem is depicted in Fig. 6.6.1. Number within circle indicates the order in which the node is generated, and number of edge indicates the city being visited.



### Example

**Example:** Find the solution of following travelling salesman problem using branch and bound method.

$$\text{Cost Matrix} = \begin{array}{|c|c|c|c|c|} \hline \infty & 20 & 30 & 10 & 11 \\ \hline 15 & \infty & 16 & 4 & 2 \\ \hline 3 & 5 & \infty & 2 & 4 \\ \hline 19 & 6 & 18 & \infty & 3 \\ \hline 16 & 4 & 7 & 16 & \infty \\ \hline \end{array}$$

### Solution:

- The procedure for dynamic reduction is as follow:
- Draw state space tree with optimal reduction cost at root node.
- Derive cost of path from node i to j by setting all entries in  $i^{\text{th}}$  row and  $j^{\text{th}}$  column as  $\infty$ . Set  $M[j][i] = \infty$
- Cost of corresponding node N for path i to j is summation of optimal cost + reduction cost +  $M[j][i]$



- After exploring all nodes at level i, set node with minimum cost as E node and repeat the procedure until all nodes are visited.
- Given matrix is not reduced. In order to find reduced matrix of it, we will first find the row reduced matrix followed by column reduced matrix if needed. We can find row reduced matrix by subtracting minimum element of each row from each element of corresponding row. Procedure is described below:
  - Reduce above cost matrix by subtracting minimum value from each row and column.

|          |          |          |          |          |                  |              |              |              |              |              |          |
|----------|----------|----------|----------|----------|------------------|--------------|--------------|--------------|--------------|--------------|----------|
| $\infty$ | 20       | 30       | 10       | 11       | $\rightarrow 10$ | $\infty$     | 10           | 20           | 0            | 1            |          |
| 15       | $\infty$ | 16       | 4        | 2        | $\rightarrow 2$  | 13           | $\infty$     | 14           | 2            | 0            |          |
| 3        | 5        | $\infty$ | 2        | 4        | $\rightarrow 2$  | 1            | 3            | $\infty$     | 0            | 2            | $= M'_1$ |
| 19       | 6        | 18       | $\infty$ | 3        | $\rightarrow 3$  | 16           | 3            | 15           | $\infty$     | 0            |          |
| 16       | 4        | 7        | 16       | $\infty$ | $\rightarrow 4$  | 12           | 0            | 3            | 12           | $\infty$     |          |
|          |          |          |          |          |                  | $\downarrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ |          |
|          |          |          |          |          |                  | 1            | 0            | 3            | 0            | 0            |          |

$M'_1$

is not reduced matrix. Reduce it subtracting minimum value from corresponding column. Doing this we get,

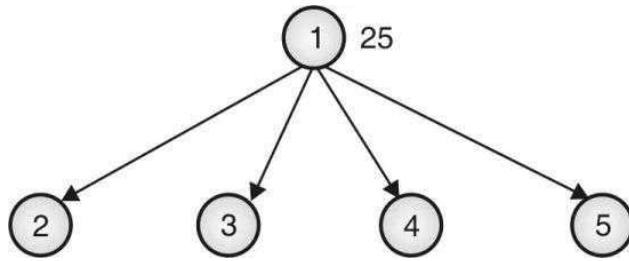
|          |          |          |          |          |         |
|----------|----------|----------|----------|----------|---------|
| $\infty$ | 10       | 17       | 0        | 1        |         |
| 12       | $\infty$ | 11       | 2        | 0        |         |
| 0        | 3        | $\infty$ | 0        | 2        | $= M_1$ |
| 15       | 3        | 12       | $\infty$ | 0        |         |
| 11       | 0        | 0        | 12       | $\infty$ |         |

Cost of  $M_1 = C(1)$

= Row reduction cost + Column reduction cost  
 =  $(10 + 2 + 2 + 3 + 4) + (1 + 3) = 25$

This means all tours in graph has length at least 25. This is the optimal cost of the path.

**State space tree**



Let us find cost of edge from node 1 to 2, 3, 4, 5.

**Select edge 1-2:**

Set  $M_1[1][ ] = M_1[ ][2] =$

$\infty$  Set  $M_1[2][1] = \infty$

Reduce the resultant matrix if required.

|              |              |              |              |              |                       |
|--------------|--------------|--------------|--------------|--------------|-----------------------|
| $\infty$     | $\infty$     | $\infty$     | $\infty$     | $\infty$     | $\rightarrow x$       |
| $\infty$     | $\infty$     | 11           | 2            | 0            | $\rightarrow 0$       |
| 0            | $\infty$     | $\infty$     | 0            | 2            | $\rightarrow 0 = M_2$ |
| 15           | $\infty$     | 12           | $\infty$     | 0            | $\rightarrow 0$       |
| 11           | $\infty$     | 0            | 12           | $\infty$     | $\rightarrow 0$       |
| $\downarrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ |                       |
| 0            | x            | 0            | 0            | 0            |                       |

$M_2$  is already reduced.

Cost of node 2 :

$C(2) = C(1) + \text{Reduction cost} + M_1[1][2]$

$= 25 + 0 + 10 = 35$

**Select edge 1-3**

Set  $M_1[1][ ] = M_1[ ][3] =$

$\infty$  Set  $M_1[3][1] = \infty$

Reduce the resultant matrix if required.

|              |              |              |              |              |                             |
|--------------|--------------|--------------|--------------|--------------|-----------------------------|
| $\infty$     | $\infty$     | $\infty$     | $\infty$     | $\infty$     | $\rightarrow x$             |
| 12           | $\infty$     | 11           | 2            | 0            | $\rightarrow 0$             |
| 0            | $\infty$     | $\infty$     | 0            | 2            | $\rightarrow 0 \Rightarrow$ |
| 15           | $\infty$     | 12           | $\infty$     | 0            | $\rightarrow 0$             |
| 11           | $\infty$     | 0            | 12           | $\infty$     | $\rightarrow 0$             |
| $\downarrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ |                             |
| 11           | 0            | x            | 0            | 0            |                             |

|          |          |          |          |          |
|----------|----------|----------|----------|----------|
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| 1        | $\infty$ | $\infty$ | 2        | 0        |
| $\infty$ | 3        | $\infty$ | 0        | 2        |
| 4        | 3        | $\infty$ | $\infty$ | 0        |
| 0        | 0        | $\infty$ | 12       | $\infty$ |

$= M_3$

Cost of node 3:

$C(3) = C(1) + \text{Reduction cost} + M_1[1][3]$

$= 25 + 11 + 17 = 53$

**Select edge 1-4:**

Set  $M_1[1][ ] = M_1[ ][4]$

$= \infty$  Set  $M_1[4][1] = \infty$

Reduce resultant matrix if required.

|              |              |              |              |              |                 |
|--------------|--------------|--------------|--------------|--------------|-----------------|
| $\infty$     | $\infty$     | $\infty$     | $\infty$     | $\infty$     | $\rightarrow x$ |
| 12           | $\infty$     | 11           | $\infty$     | 0            | $\rightarrow 0$ |
| 0            | 3            | $\infty$     | $\infty$     | 2            | $\rightarrow 0$ |
| $\infty$     | 3            | 12           | $\infty$     | 0            | $\rightarrow 0$ |
| 11           | 0            | 0            | $\infty$     | $\infty$     | $\rightarrow 0$ |
| $\downarrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ |                 |
| 0            | 0            | 0            | x            | 0            |                 |

$M_1 \Rightarrow$   $M_4$

Matrix  $M_4$  is already reduced. Cost of node 4:

$$C(4) = C(1) + \text{Reduction cost} + M_1[1][4] \\ = 25 + 0 + 0 = 25$$

**Select edge 1-5:**

Set  $M_1[1][ ] = M_1[ ][5]$

$= \infty$  Set  $M_1[5][1] = \infty$

Reduce the resultant matrix if required.

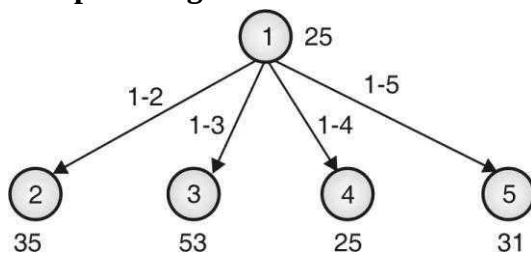
|              |              |              |              |              |                 |
|--------------|--------------|--------------|--------------|--------------|-----------------|
| $\infty$     | $\infty$     | $\infty$     | $\infty$     | $\infty$     | $\rightarrow x$ |
| 12           | $\infty$     | 11           | 2            | $\infty$     | $\rightarrow 2$ |
| 0            | 3            | $\infty$     | 0            | $\infty$     | $\rightarrow 0$ |
| 15           | 3            | 12           | $\infty$     | $\infty$     | $\rightarrow 3$ |
| $\infty$     | 0            | 0            | 12           | $\infty$     | $\rightarrow 0$ |
| $\downarrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ |                 |
| 0            | 0            | 0            | 0            | x            |                 |

$M_1 \Rightarrow$   $M_5$

Cost of node 5:

$$C(5) = C(1) + \text{reduction cost} + M_1[1][5] \\ = 25 + 5 + 1 = 31$$

**State space diagram:**



Node 4 has minimum cost for path 1-4. We can go to vertex 2, 3 or 5. Let's explore all three nodes.

**Select path 1-4-2 : (Add edge 4-2)**

Set  $M_4[1][ ] = M_4[4][ ] = M_4[ ][ ]$

$[2] = \infty$  Set  $M_4[2][1] = \infty$

Reduce resultant matrix if required.

|              |              |              |              |              |                 |
|--------------|--------------|--------------|--------------|--------------|-----------------|
| $\infty$     | $\infty$     | $\infty$     | $\infty$     | $\infty$     | $\rightarrow x$ |
| $\infty$     | $\infty$     | 11           | $\infty$     | 0            | $\rightarrow 0$ |
| 0            | $\infty$     | $\infty$     | $\infty$     | 2            | $\rightarrow 0$ |
| $\infty$     | $\infty$     | $\infty$     | $\infty$     | $\infty$     | $\rightarrow x$ |
| 11           | $\infty$     | 0            | $\infty$     | $\infty$     | $\rightarrow 0$ |
| $\downarrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ |                 |
| 0            | 0            | 0            | x            | 0            |                 |

$M_4 \Rightarrow$   $M_6$



Matrix  $M_6$  is already reduced. Cost of node 6:

$$C(6) = C(4) + \text{Reduction cost} + M_4[4][2]$$

$$= 25 + 0 + 3 = 28$$

**Select edge 4-3 (Path 1-4-3):**

$$\text{Set } M_4[1][ ] = M_4[4][ ] = M_4[ ][3] =$$

$$\infty \text{ Set } M[3][1] = \infty$$

Reduce the resultant matrix if required.

$$M_4 \Rightarrow \begin{array}{|c|c|c|c|c|} \hline \infty & \infty & \infty & \infty & \infty \\ \hline 12 & \infty & \infty & \infty & 0 \\ \hline \infty & 3 & \infty & \infty & 2 \\ \hline \infty & \infty & \infty & \infty & \infty \\ \hline 11 & 0 & \infty & \infty & \infty \\ \hline \end{array} \begin{array}{l} \rightarrow x \\ \rightarrow 0 \\ \rightarrow 2 \\ \rightarrow \infty \\ \rightarrow 0 \end{array} \Rightarrow \begin{array}{|c|c|c|c|c|} \hline \infty & \infty & \infty & \infty & \infty \\ \hline 12 & \infty & \infty & \infty & 0 \\ \hline \infty & 1 & \infty & \infty & 0 \\ \hline \infty & \infty & \infty & \infty & \infty \\ \hline 11 & 0 & \infty & \infty & \infty \\ \hline \end{array} = M'_7$$

$$\downarrow \downarrow \downarrow \downarrow \downarrow$$

$$11 \ 0 \ x \ x \ 0$$

$M'_7$  is not reduced. Reduce it by subtracting 11 from column 1.

$$\therefore M'_7 \Rightarrow \begin{array}{|c|c|c|c|c|} \hline \infty & \infty & \infty & \infty & \infty \\ \hline 1 & \infty & \infty & \infty & 0 \\ \hline \infty & 1 & \infty & \infty & 2 \\ \hline \infty & \infty & \infty & \infty & \infty \\ \hline 0 & 0 & \infty & \infty & \infty \\ \hline \end{array} = M_7$$

Cost of node 7:

$$C(7) = C(4) + \text{Reduction cost} + M_4[4][3]$$

$$= 25 + 2 + 11 + 12 = 50$$

**Select edge 4-5 (Path 1-4-5):**

$$M_4 \Rightarrow \begin{array}{|c|c|c|c|c|} \hline \infty & \infty & \infty & \infty & \infty \\ \hline 12 & \infty & 11 & \infty & \infty \\ \hline 0 & 3 & \infty & \infty & \infty \\ \hline \infty & \infty & \infty & \infty & \infty \\ \hline \infty & 0 & 0 & \infty & \infty \\ \hline \end{array} \begin{array}{l} \rightarrow x \\ \rightarrow 11 \\ \rightarrow 0 \\ \rightarrow x \\ \rightarrow 0 \end{array} \Rightarrow \begin{array}{|c|c|c|c|c|} \hline \infty & \infty & \infty & \infty & \infty \\ \hline 1 & \infty & 0 & \infty & \infty \\ \hline 0 & 3 & \infty & \infty & \infty \\ \hline \infty & \infty & \infty & \infty & \infty \\ \hline \infty & 0 & 0 & \infty & \infty \\ \hline \end{array} = M_8$$

$$\downarrow \downarrow \downarrow \downarrow \downarrow$$

$$0 \ 0 \ 0 \ x \ x$$

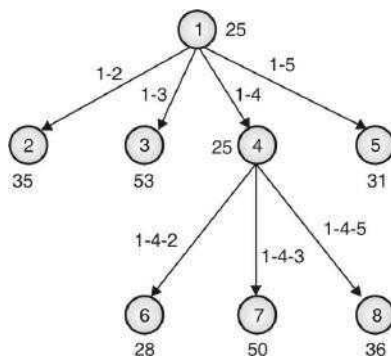
Matrix  $M_8$  is reduced.

Cost of node 8:

$$C(8) = C(4) + \text{Reduction cost} + M_4[4][5]$$

$$= 25 + 11 + 0 = 36$$

**State space tree**



Path 1-4-2 leads to minimum cost. Let's find the cost for two possible paths.

### Add edge 2-3 (Path 1-4-2-3):

Set  $M_6[1][ ] = M_6[4][ ] = M_6[2][ ]$

$= M_6[ ][3] = \infty$

Set  $M_6[3][1] = \infty$

Reduce resultant matrix if required.

$$M_6 \Rightarrow \begin{array}{|c|c|c|c|c|c|} \hline \infty & \infty & \infty & \infty & \infty & \\ \hline \infty & \infty & \infty & \infty & \infty & \\ \hline 0 & \infty & \infty & \infty & 2 & \\ \hline \infty & \infty & \infty & \infty & \infty & \\ \hline 11 & \infty & \infty & \infty & \infty & \\ \hline \end{array} \begin{array}{l} \rightarrow x \\ \rightarrow x \\ \rightarrow 0 \\ \rightarrow x \\ \rightarrow 11 \end{array} \Rightarrow \begin{array}{|c|c|c|c|c|c|} \hline \infty & \infty & \infty & \infty & \infty & \\ \hline \infty & \infty & \infty & \infty & \infty & \\ \hline 0 & \infty & \infty & \infty & 2 & \\ \hline \infty & \infty & \infty & \infty & \infty & \\ \hline 0 & \infty & \infty & \infty & \infty & \\ \hline \end{array} = M'_9$$

$\downarrow \downarrow \downarrow \downarrow \downarrow$   
 $0 \quad x \quad x \quad x \quad 2$

$$\therefore M'_9 \Rightarrow \begin{array}{|c|c|c|c|c|} \hline \infty & \infty & \infty & \infty & \infty \\ \hline \infty & \infty & \infty & \infty & \infty \\ \hline 0 & \infty & \infty & \infty & 0 \\ \hline \infty & \infty & \infty & \infty & \infty \\ \hline 0 & \infty & \infty & \infty & \infty \\ \hline \end{array} = M_9$$

Cost of node 9:

$C(9) = C(6) + \text{Reduction cost} + M_6[2][3]$

$= 28 + 11 + 2 + 11 = 52$

### Add edge 2-5 (Path 1-4-2-5):

Set  $M_6[1][ ] = M_6[4][ ] = M_6[2][ ] = M_6[ ][5]$

$= \infty$  Set  $M_6[5][1] = \infty$

Reduce resultant matrix if required.

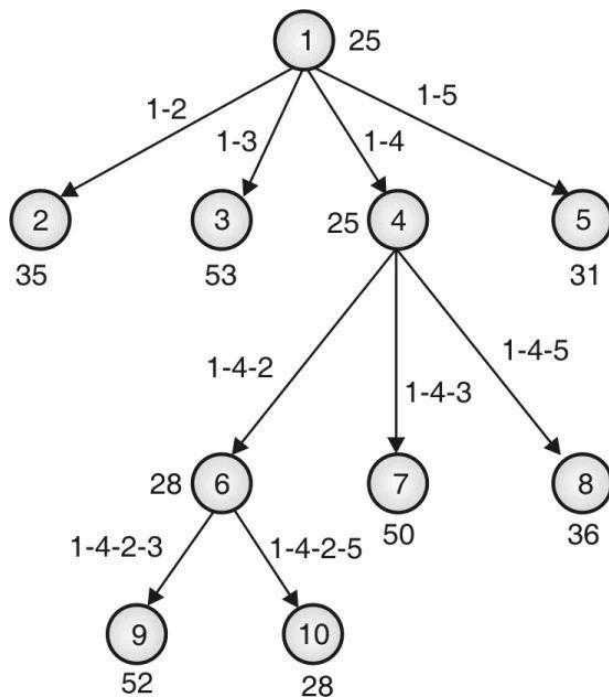
$$\therefore M_6 \Rightarrow \begin{array}{|c|c|c|c|c|} \hline \infty & \infty & \infty & \infty & \infty \\ \hline \infty & \infty & \infty & \infty & \infty \\ \hline 0 & \infty & \infty & \infty & \infty \\ \hline \infty & \infty & \infty & \infty & \infty \\ \hline \infty & \infty & 0 & \infty & \infty \\ \hline \end{array} = M_{10}$$

Cost of node 10:

$$C(10) = C(6) + \text{Reduction cost} + M_6 [2][5]$$

$$= 28 + 0 + 0 = 28$$

**State space tree**



**Add edge 5-3 (Path 1-4-2-5-3):**

$$\therefore M_{10} \Rightarrow \begin{array}{|c|c|c|c|c|} \hline \infty & \infty & \infty & \infty & \infty \\ \hline \infty & \infty & \infty & \infty & \infty \\ \hline \infty & \infty & \infty & \infty & \infty \\ \hline \infty & \infty & \infty & \infty & \infty \\ \hline \infty & \infty & \infty & \infty & \infty \\ \hline \end{array} = M_{11}$$

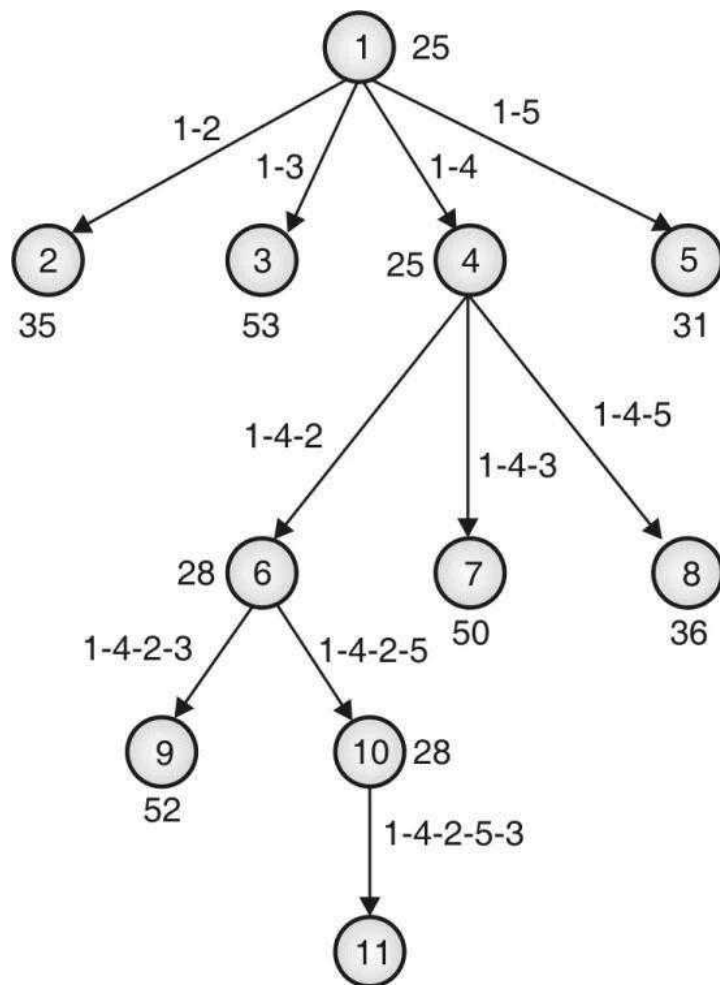
Cost of node 11:

$$C(11) = C(10) + \text{Reduction cost} + M_{10} [5][3]$$

$$= 28 + 0 + 0 = 28$$



**State space tree:**



So we can select any of the edge. Thus the final path includes the edges  $\langle 3, 1 \rangle$ ,  $\langle 5, 3 \rangle$ ,  $\langle 1, 4 \rangle$ ,  $\langle 4, 2 \rangle$ ,  $\langle 2, 5 \rangle$ , that forms the path  $1 - 4 - 2 - 5 - 3 - 1$ . This path has cost of 28.

## UNIT 5

### Tractable and Intractable Problems

Tractable problems refer to computational problems that can be solved efficiently using algorithms that can scale with the input size of the problem. In other words, the time required to solve a tractable problem increases at most polynomial with the input size.

On the other hand, intractable problems are computational problems for which no known algorithm can solve them efficiently in the worst-case scenario. This means that the time required to solve an intractable problem grows exponentially or even faster with the input size.

One example of a tractable problem is computing the sum of a list of  $n$  numbers. The time required to solve this problem scales linearly with the input size, as each number can be added to a running total in constant time. Another example is computing the shortest path between two nodes in a graph, which can be solved efficiently using algorithms like Dijkstra's algorithm or the A\* algorithm.

In contrast, some well-known intractable problems include the traveling salesman problem, the knapsack problem, and the Boolean satisfiability problem. These problems are NP-hard, meaning that any problem in NP (the set of problems that can be solved in polynomial time using a non-deterministic Turing machine) can be reduced to them in polynomial time. While it is possible to find approximate solutions to these problems, there is no known algorithm that can solve them exactly in polynomial time.

In summary, tractable problems are those that can be solved efficiently with algorithms that scale well with the input size, while intractable problems are those that cannot be solved efficiently in the worst-case scenario.

### Examples of Tractable problems

1. **Sorting:** Given a list of  $n$  items, the task is to sort them in ascending or descending order. Algorithms like Quick Sort and Merge Sort can solve this problem in  $O(n \log n)$  time complexity.
2. **Matrix multiplication:** Given two matrices  $A$  and  $B$ , the task is to find their product  $C = AB$ . The best-known algorithm for matrix multiplication runs in  $O(n^{2.37})$  time complexity, which is considered tractable for practical applications.
3. **Shortest path in a graph:** Given a graph  $G$  and two nodes  $s$  and  $t$ , the task is to find the shortest path between  $s$  and  $t$ . Algorithms like Dijkstra's algorithm and the A\* algorithm can solve this problem in  $O(m + n \log n)$  time complexity, where  $m$  is the number of edges and  $n$  is the number of nodes in the graph.
4. **Linear programming:** Given a system of linear constraints and a linear objective function, the task is to find the values of the variables that optimize the objective function subject to the constraints. Algorithms like the simplex method can solve this problem in polynomial time.
5. **Graph coloring:** Given an undirected graph  $G$ , the task is to assign a color to each node such that no two adjacent nodes have the same color, using as few colors as possible. The greedy algorithm can solve this problem in  $O(n^2)$  time complexity, where  $n$  is the number of nodes in the graph.

These problems are considered tractable because algorithms exist that can solve them in polynomial time complexity, which means that the time required to solve them grows no faster than a polynomial function of the input size.

### Examples of intractable problems

1. Traveling salesman problem (TSP): Given a set of cities and the distances between them, the task is to find the shortest possible route that visits each city exactly once and returns to the starting city. The best-known algorithms for solving the TSP have an exponential worst-case time complexity, which makes it intractable for large instances of the problem.
2. Knapsack problem: Given a set of items with weights and values, and a knapsack that can carry a maximum weight, the task is to find the most valuable subset of items that can be carried by the knapsack. The knapsack problem is also NP-hard and is intractable for large instances of the problem.
3. Boolean satisfiability problem (SAT): Given a boolean formula in conjunctive normal form (CNF), the task is to determine if there exists an assignment of truth values to the variables that makes the formula true. The SAT problem is one of the most well-known NP-complete problems, which means that any NP problem can be reduced to SAT in polynomial time.
4. Subset sum problem: Given a set of integers and a target sum, the task is to find a subset of the integers that sums up to the target sum. Like the knapsack problem, the subset sum problem is also intractable for large instances of the problem.

### P (Polynomial) problems

P problems refer to problems where an algorithm would take a polynomial amount of time to solve, or where Big-O is a polynomial (i.e.  $O(1)$ ,  $O(n)$ ,  $O(n^2)$ , etc). These are problems that would be considered 'easy' to solve, and thus do not generally have immense run times.

### NP (Non-deterministic Polynomial) Problems

NP problems were a little harder for me to understand, but I think this is what they are. In terms of solving a NP problem, the run-time would not be polynomial. It would be something like  $O(n!)$  or something much larger.

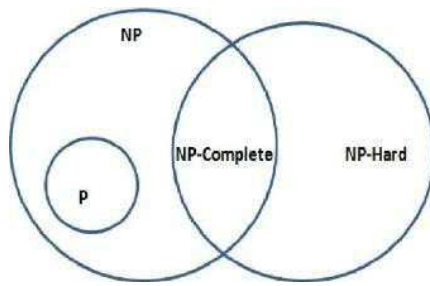
### NP-Hard Problems

A problem is classified as NP-Hard when an algorithm for solving it can be translated to solve *any* NP problem. Then we can say, this problem is *at least* as hard as any NP problem, but it could be much harder or more complex.

### NP-Complete Problems

NP-Complete problems are problems that live in both the NP and NP-Hard classes. This means that NP-Complete problems can be verified in polynomial time and that any NP problem can be reduced to this problem in polynomial time.





## Bin Packing problem

Bin Packing problem involves assigning  $n$  items of different weights and bins each of capacity  $c$  to a bin such that number of total used bins is minimized. It may be assumed that all items have weights smaller than bin capacity.

The following 4 algorithms depend on the order of their inputs. They pack the item given first and then move on to the next input or next item

### 1) Next Fit algorithm

The simplest approximate approach to the bin packing problem is the Next-Fit (NF) algorithm which is explained later in this article. The first item is assigned to bin 1. Items 2,...,  $n$  are then considered by increasing indices : each item is assigned to the current bin, if it fits; otherwise, it is assigned to a new bin, which becomes the current one.

### Visual Representation

Let us consider the same example as used above and bins of size 1



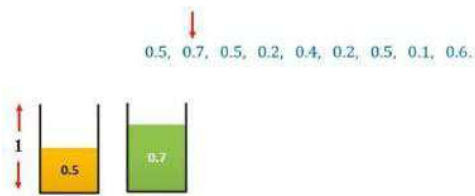
Assuming the sizes of the items be  $\{0.5, 0.7, 0.5, 0.2, 0.4, 0.2, 0.5, 0.1, 0.6\}$ .

The minimum number of bins required would be  $\text{Ceil}((\text{Total Weight}) / (\text{Bin Capacity})) = \text{Ceil}(3.7/1) = 4$  bins.

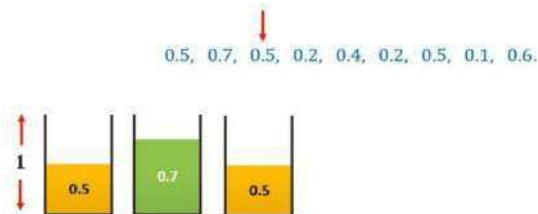
The Next fit solution (NF(I))for this instance I would be- Considering 0.5 sized item first, we can place it in the first bin



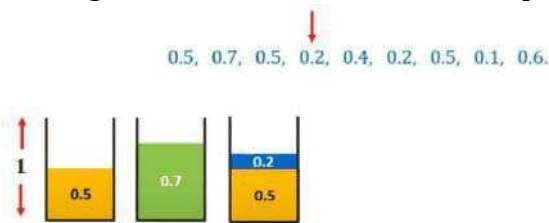
Moving on to the 0.7 sized item, we cannot place it in the first bin. Hence we place it in a new bin.



Moving on to the 0.5 sized item, we cannot place it in the current bin. Hence we place it in a new bin.



Moving on to the 0.2 sized item, we can place it in the current (third bin)



Similarly, placing all the other items following the Next-Fit algorithm we get-



Thus we need 6 bins as opposed to the 4 bins of the optimal solution. Thus we can see that this algorithm is not very efficient.

### Analyzing the approximation ratio of Next-Fit algorithm

The time complexity of the algorithm is clearly  $O(n)$ . It is easy to prove that, for any instance  $I$  of BPP, the solution value  $NF(I)$  provided by the algorithm satisfies the bound

$$NF(I) < 2z(I)$$

where  $z(I)$  denotes the optimal solution value. Furthermore, there exist instances for which the ratio  $NF(I)/z(I)$  is arbitrarily close to 2, i.e. the worst-case approximation ratio of NF is  $r(NF) = 2$ .

### Pseudocode

NEXT FIT (size[], n, c)

size[] is the array containing the sizes of the items, n is the number of items and c is the capacity of the bin

{

    Initialize result (Count of bins) and remaining capacity in current bin. res = 0

    bin\_rem = c

    Place items one by one for (int i = 0; i < n; i++) {

```

// If this item can't fit in current bin if (size[i] > bin_rem)
{
    Use a new bin res++
    bin_rem = c - size[i]
}
else
    bin_rem -= size[i];
}
return res;
}

```

## 2) First Fit algorithm

A better algorithm, First-Fit (FF), considers the items according to increasing indices and assigns each item to the lowest indexed initialized bin into which it fits; only when the current item cannot fit into any initialized bin, is a new bin introduced

### Visual Representation

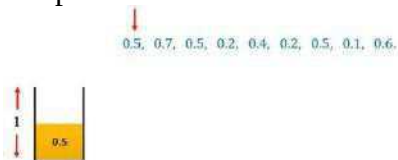
Let us consider the same example as used above and bins of size 1



Assuming the sizes of the items be {0.5, 0.7, 0.5, 0.2, 0.4, 0.2, 0.5, 0.1, 0.6}.

The minimum number of bins required would be  $\text{Ceil}((\text{Total Weight}) / (\text{Bin Capacity})) = \text{Ceil}(3.7/1) = 4$  bins.

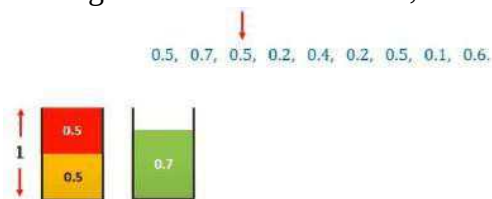
The First fit solution (FF(I)) for this instance I would be- Considering 0.5 sized item first, we can place it in the first bin



Moving on to the 0.7 sized item, we cannot place it in the first bin. Hence we place it in a new bin.

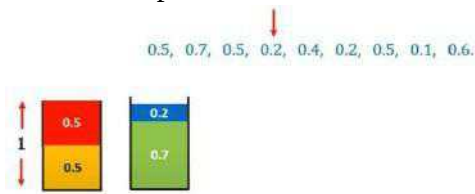


Moving on to the 0.5 sized item, we can place it in the first bin.

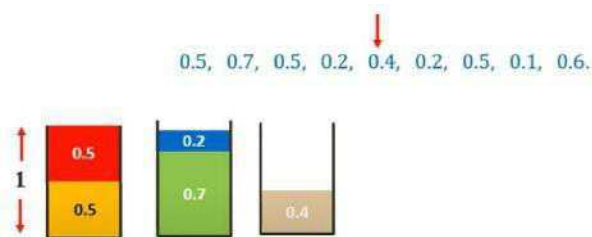




Moving on to the 0.2 sized item, we can place it in the first bin, we check with the second bin and we can place it there.



Moving on to the 0.4 sized item, we cannot place it in any existing bin. Hence we place it in a new bin.



Similarly, placing all the other items following the First-Fit algorithm we get-



Thus we need 5 bins as opposed to the 4 bins of the optimal solution but is much more efficient than Next-Fit algorithm.

### Analyzing the approximation ratio of Next-Fit algorithm

If  $FF(I)$  is the First-fit implementation for  $I$  instance and  $z(I)$  is the most optimal solution, then:

$$FF(I) \leq \frac{17}{10} z(I) + 2$$

for all instances  $I$  of BPP, and that there exist instances  $I$ , with  $z(I)$  arbitrarily large, for which

$$FF(I) > \frac{17}{10} z(I) - 8.$$

It can be seen that the First Fit never uses more than  $1.7 * z(I)$  bins. So First-Fit is better than Next Fit in terms of upper bound on number of bins.

### Pseudocode

FIRSTFIT(size[], n, c)

{

size[] is the array containing the sizes of the items, n is the number of items and c is the capacity of the bin

/Initialize result (Count of bins)

```

res = 0;
Create an array to store remaining space in bins there can be at most n bins bin_rem[n];

Place items one by one for (int i = 0; i < n; i++) {
    Find the first bin that can accommodate weight[i] int j;
    for (j = 0; j < res; j++) {
        if (bin_rem[j] >= size[i]) { bin_rem[j] = bin_rem[j] - size[i]; break;
        }
    }

    If no bin could accommodate size[i] if (j == res) {
        bin_rem[res] = c - size[i]; res++;
    }

}
return res;
}

```

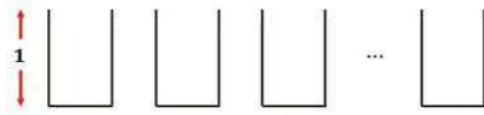
### 3) Best Fit Algorithm

The next algorithm, Best-Fit (BF), is obtained from FF by assigning the current item to the feasible bin (if any) having the smallest residual capacity (breaking ties in favor of the lowest indexed bin).

Simply put, the idea is to place the next item in the *tightest* spot. That is, put it in the bin so that the smallest empty space is left.

### Visual Representation

Let us consider the same example as used above and bins of size 1

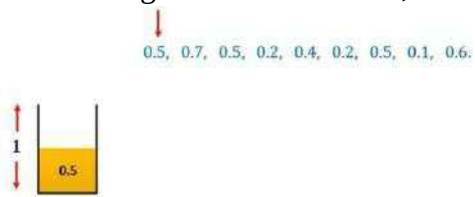


Assuming the sizes of the items be {0.5, 0.7, 0.5, 0.2, 0.4, 0.2, 0.5, 0.1, 0.6}.

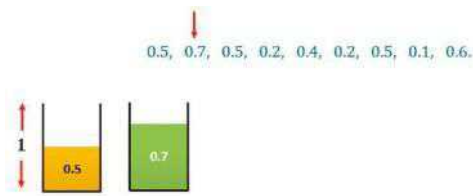
The minimum number of bins required would be  $\text{Ceil}((\text{Total Weight}) / (\text{Bin Capacity})) = \text{Ceil}(3.7/1) = 4$  bins.

The First fit solution (FF(I)) for this instance I would be-

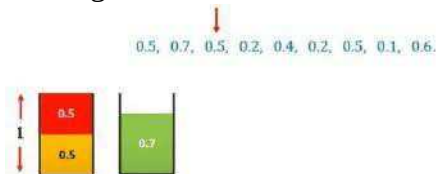
Considering 0.5 sized item first, we can place it in the first bin



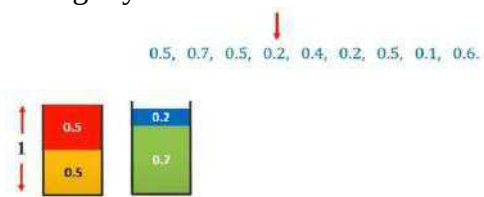
Moving on to the 0.7 sized item, we cannot place it in the first bin. Hence we place it in a new bin.



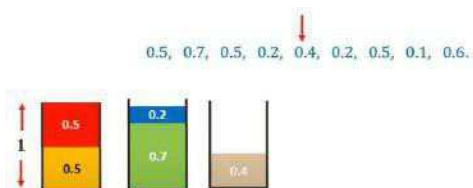
Moving on to the 0.5 sized item, we can place it in the first bin tightly.



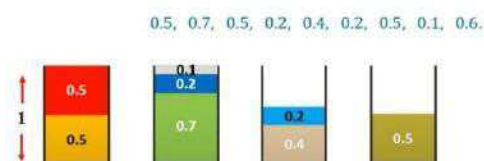
Moving on to the 0.2 sized item, we cannot place it in the first bin but we can place it in second bin tightly.



Moving on to the 0.4 sized item, we cannot place it in any existing bin. Hence we place it in a new bin.



Similarly, placing all the other items following the First-Fit algorithm we get-



Thus we need 5 bins as opposed to the 4 bins of the optimal solution but is much more efficient than Next-Fit algorithm.

### Analyzing the approximation ratio of Best-Fit algorithm

It can be noted that Best-Fit (BF), is obtained from FF by assigning the current item to the feasible bin (if any) having the smallest residual capacity (breaking ties in favour of the lowest indexed bin). BF satisfies the same worst-case bounds as FF

### Analysis Of upper-bound of Best-Fit algorithm

If  $z(I)$  is the optimal number of bins, then Best Fit never uses more than  $2 * z(I) - 2$  bins. So Best Fit is same as Next Fit in terms of upper bound on number of bins.

### Pseudocode

BESTFIT(size[], n, c)

{

size[] is the array containing the sizes of the items, n is the number of items and c is the capacity of the bin

Initialize result (Count of bins) res = 0;

Create an array to store remaining space in bins there can be at most n bins bin\_rem[n];

Place items one by one for (int i = 0; i < n; i++) {

Find the best bin that can accommodate weight[i] int j;

Initialize minimum space left and index of best bin int min = c + 1, bi = 0;

for (j = 0; j < res; j++) {

if (bin\_rem[j] >= size[i] && bin\_rem[j] - size[i] < min) { bi = j;

min = bin\_rem[j] - size[i];

}

}

If no bin could accommodate weight[i], create a new bin if (min == c + 1) {

bin\_rem[res] = c - size[i]; res++;

}

else

Assign the item to best bin bin\_rem[bi] -= size[i];

}

return res;

}

### Approximation Algorithms for the Traveling Salesman Problem

We solved the traveling salesman problem by exhaustive search in Section 3.4, mentioned its decision version as one of the most well-known *NP*-complete problems in Section 11.3, and saw how its instances can be solved by a branch-and-bound algorithm in Section 12.2. Here, we consider several approximation algorithms, a small sample of dozens of such algorithms suggested over the years for this famous problem.

But first let us answer the question of whether we should hope to find a polynomial-time approximation algorithm with a finite performance ratio on all instances of the traveling salesman problem. As the following theorem [Sah76] shows, the answer turns out to be no, unless  $P = NP$ .

**THEOREM 1** If  $P \neq NP$ , there exists no  $c$ -approximation algorithm for the traveling salesman problem, i.e., there exists no polynomial-time approximation algorithm for this problem so that for all instances



$$f(s_a) \leq cf(s^*)$$

for some constant  $c$ .

### Nearest-neighbour algorithm

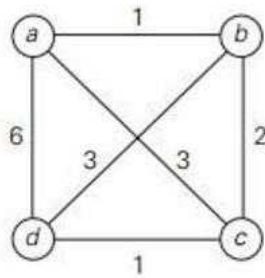
The following well-known greedy algorithm is based on the **nearest-neighbor** heuristic: always go next to the nearest unvisited city.

**Step 1** Choose an arbitrary city as the start.

**Step 2** Repeat the following operation until all the cities have been visited: go to the unvisited city nearest the one visited last (ties can be broken arbitrarily).

**Step 3** Return to the starting city.

**EXAMPLE 1** For the instance represented by the graph in Figure 12.10, with  $a$  as the starting vertex, the nearest-neighbor algorithm yields the tour (Hamiltonian circuit)  $s_a$ :  $a - b - c - d - a$  of length 10.



The optimal solution, as can be easily checked by exhaustive search, is the tour  $s^*$ :  $a - b - d - c - a$  of length 8. Thus, the accuracy ratio of this approximation is

$$r(s_a) = \frac{f(s_a)}{f(s^*)} = \frac{10}{8} = 1.25$$

(i.e., tour  $s_a$  is 25% longer than the optimal tour  $s^*$ ).

Unfortunately, except for its simplicity, not many good things can be said about the nearest-neighbor algorithm. In particular, nothing can be said in general about the accuracy of solutions obtained by this algorithm because it can force us to traverse a very long edge on the last leg of the tour. Indeed, if we change the weight of edge  $(a, d)$  from 6 to an arbitrary large number  $w \geq 6$  in Example 1, the algorithm will still yield the tour  $a - b - c - d - a$  of length  $4 + w$ , and the optimal solution will still be  $a - b - d - c - a$  of length 8. Hence,

$$r(s_a) = \frac{f(s_a)}{f(s^*)} = \frac{4 + w}{8},$$

which can be made as large as we wish by choosing an appropriately large value of  $w$ . Hence,

$R_A = \infty$  for this algorithm (as it should be according to Theorem 1).

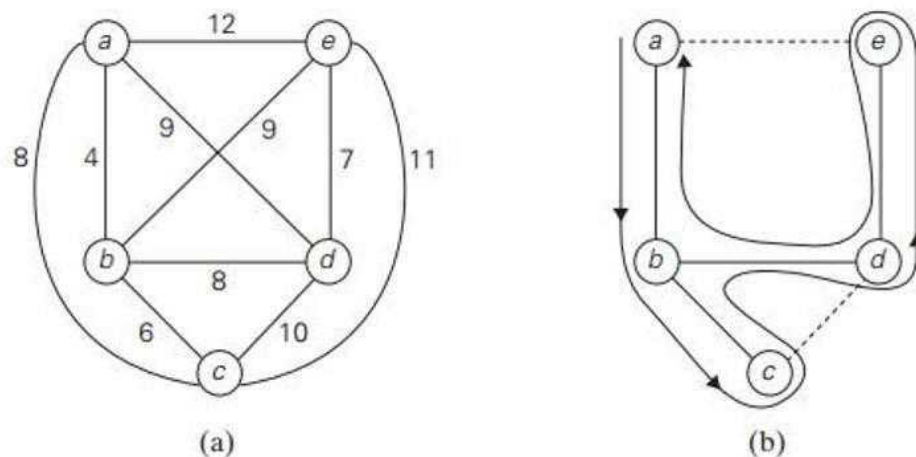
### Twice-around-the-tree algorithm

**Step 1** Construct a minimum spanning tree of the graph corresponding to a given instance of the traveling salesman problem.

**Step 2** Starting at an arbitrary vertex, perform a walk around the minimum spanning tree recording all the vertices passed by. (This can be done by a DFS traversal.)

**Step 3** Scan the vertex list obtained in Step 2 and eliminate from it all repeated occurrences of the same vertex except the starting one at the end of the list. (This step is equivalent to making shortcuts in the walk.) The vertices remaining on the list will form a Hamiltonian circuit, which is the output of the algorithm.

**EXAMPLE 2** Let us apply this algorithm to the graph in Figure 12.11a. The minimum spanning tree of this graph is made up of edges  $(a, b)$ ,  $(b, c)$ ,  $(b, d)$ , and  $(d, e)$ . A twice-



around-the-tree walk that starts and ends at  $a$  is

$a, b, c, b, d, e, d, b, a$ .

Eliminating the second  $b$  (a shortcut from  $c$  to  $d$ ), the second  $d$ , and the third  $b$  (a shortcut from  $e$  to  $a$ ) yields the Hamiltonian circuit

$a, b, c, d, e, a$

of length 39.

The tour obtained in Example 2 is not optimal. Although that instance is small enough to find an optimal solution by either exhaustive search or branch-and-bound, we refrained from doing so to reiterate a general point. As a rule, we do not know what the length of an optimal tour actually is, and therefore we cannot compute the accuracy ratio  $f(s_a)/f(s^*)$ . For the twice-around-the-tree algorithm, we can at least estimate it above, provided the graph is Euclidean.

**Fermat's Little Theorem:**

If  $n$  is a prime number, then for every  $a$ ,  $1 < a < n-1$ ,

$$a^{n-1} \equiv 1 \pmod{n} \text{ OR}$$

$$a^{n-1} \% n = 1$$

Example: Since 5 is prime,  $2^4 \equiv 1 \pmod{5}$  [or  $2^4 \% 5 = 1$ ],

$$3^4 \equiv 1 \pmod{5} \text{ and } 4^4 \equiv 1 \pmod{5}$$

Since 7 is prime,  $2^6 \equiv 1 \pmod{7}$ ,

$$3^6 \equiv 1 \pmod{7}, 4^6 \equiv 1 \pmod{7}$$

$$5^6 \equiv 1 \pmod{7} \text{ and } 6^6 \equiv 1 \pmod{7}$$

1) Repeat following  $k$  times:

- a) Pick  $a$  randomly in the range  $[2, n - 2]$
- b) If  $\gcd(a, n) \neq 1$ , then return false
- c) If  $a^{n-1} \not\equiv 1 \pmod{n}$ , then return false

2) Return true [probably prime].

Unlike merge sort, we don't need to merge the two sorted arrays. Thus Quicksort requires lesser auxiliary space than Merge Sort, which is why it is often preferred to Merge Sort.

Using a randomly generated pivot we can further improve the time complexity of Quicksort.

**Algorithm for random pivoting**

partition(arr[], lo, hi)

    pivot = arr[hi]

    i = lo // place for swapping

    for j := lo to hi - 1 do

        if arr[j] <= pivot then swap arr[i] with arr[j] i = i + 1

    swap arr[i] with arr[hi] return i

partition\_r(arr[], lo, hi)

    r = Random Number from lo to hi

    Swap arr[r] and arr[hi]

    return partition(arr, lo, hi)

    quicksort(arr[], lo, hi)

    if lo < hi

        p = partition\_r(arr, lo, hi)

        quicksort(arr, lo, p-1)

        quicksort(arr, p+1, hi)

**Finding kth smallest element**

**Problem Description:** Given an array  $A[]$  of  $n$  elements and a positive integer  $K$ , find the  $K$ th smallest element in the array. It is given that all array elements are distinct.

**For Example :**

Input :  $A[] = \{10, 3, 6, 9, 2, 4, 15, 23\}$ ,  $K = 4$

Output: 6

Input : A[] = {5, -8, 10, 37, 101, 2, 9}, K = 6

Output: 37

### Quick-Select : Approach similar to quick sort

This approach is similar to the quick sort algorithm where we use the partition on the input array recursively. But unlike quicksort, which processes both sides of the array recursively, this algorithm works on only one side of the partition. We recur for either the left or right side according to the position of pivot.

#### Solution Steps

1. Partition the array A[left .. right] into two subarrays A[left .. pos] and A[pos + 1 .. right] such that each element of A[left .. pos] is less than each element of A[pos + 1 .. right].
  2. Computes the number of elements in the subarray A[left .. pos] i.e. count = pos - left + 1
  3. if (count == K), then A[pos] is the Kth smallest element.
  4. Otherwise determines in which of the two subarrays A[left .. pos-1] and A[pos + 1 .. right] the Kth smallest element lies.
- If (count > K) then the desired element lies on the left side of the partition
  - If (count < K), then the desired element lies on the right side of the partition. Since we already know i values that are smaller than the kth smallest element of A[left .. right], the desired element is the (K - count)th smallest element of A[pos + 1 .. right].
  - Base case is the scenario of single element array i.e left ==right. return A[left] or A[right].

#### Pseudo-Code

// Original value for left = 0 and right = n-1

**int kthSmallest(int A[], int left, int right, int K)**

```
{
if (left == right)
return A[left]

    int pos = partition(A, left, right)
    count = pos - left + 1
    if ( count == K )
        return A[pos]

    else if ( count > K )

        return kthSmallest(A, left, pos-1, K)
    else
        return kthSmallest(A, pos+1, right, K-i)
}
```

**int partition(int A[], int l, int r)**

```
{
    int x = A[r]

    int i = l-1
```



```

for ( j = l to r-1 )
{
    if (A[j] <= x)

        {
            i = i + 1 swap(A[i], A[j])
        }
}
swap(A[i+1], A[r])
return i+1
}

```

### ***Complexity Analysis***

Time Complexity: The worst-case time complexity for this algorithm is  $O(n^2)$ , but it can be improved if we choose the pivot element randomly. If we randomly select the pivot, the expected time complexity would be linear,  **$O(n)$** .